

# Transport over Wireless

Advanced Mobile Communication Networks



---

Integrated Communication Systems Group  
Ilmenau University of Technology

# Content

- **Internet Protocol Suite**
  - Link Layer: Ethernet, PPP, ARP, MAC Addressing
  - Network Layer: IP, ICMP, Routing
  - Transport Layer: TCP, UDP, Port Numbers, Sockets
  - Application Layer: FTP, Telnet & Rlogin, HTTP, RTP
- **TCP**
- **TCP over Wireless**
- **Beyond TCP**

# Internet Protocol Suite

Questions to be answered:

- What is the purpose of ARP?
- What is the difference of ARP from DNS?
- How does ARP differ from DHCP?
- What is the purpose of encapsulation and decapsulation?
- Why do we need multiple headers? Why not a single one?
- What does the MTU mean? What is its purpose of the path MTU?
- What is the purpose of the different header fields in IP? What is the purpose of the checksum and the protocol field?
- What is the purpose of ports used by TCP and UDP? How does it differ from IP addresses?
- How are errors handled with UDP? Why is UDP not considered reliable?
- Why does the TCP/IP model provide different transport layer protocols?

# Content

- **Internet Protocol Suite**
  - Link Layer: Ethernet, PPP, ARP, MAC Addressing
  - Network Layer: IP, ICMP, Routing
  - Transport Layer: TCP, UDP, Port Numbers, Sockets
  - Application Layer: FTP, Telnet & Rlogin, HTTP, RTP

# Internet Protocol Suite

**TCP/IP** = the “**Internet protocol suite**” = a family of protocols for the “Internet”

Internet guesstimates 2003:

- 800 million users (x 2 each two years), 200 million permanent hosts

Standardisation:

- ISOC: Internet Society
- IAB: Internet Architecture Board
  - **IETF**: Internet Engineering Task Force: <http://www.ietf.org>
    - Standards & other informations are published as RFCs: Requests for Comments
  - IRTF: Internet Research Task Force

# Internet Protocol Suite

## Implementations:

- De-facto standard: BSD 4.x implementations (Berkeley Software Distribution)
- Subsequent versions come with new TCP features, e.g.
  - 4.3 BSD Tahoe (1988): slow start, congestion avoidance, fast retransmit
  - 4.3 BSD Reno (1990): fast recovery
- Other TCP/IP stacks derived from BSD
- **Implemented mechanisms, default parameter settings, and bugs are different on different operating systems (e.g. versions of MS Windows)!**

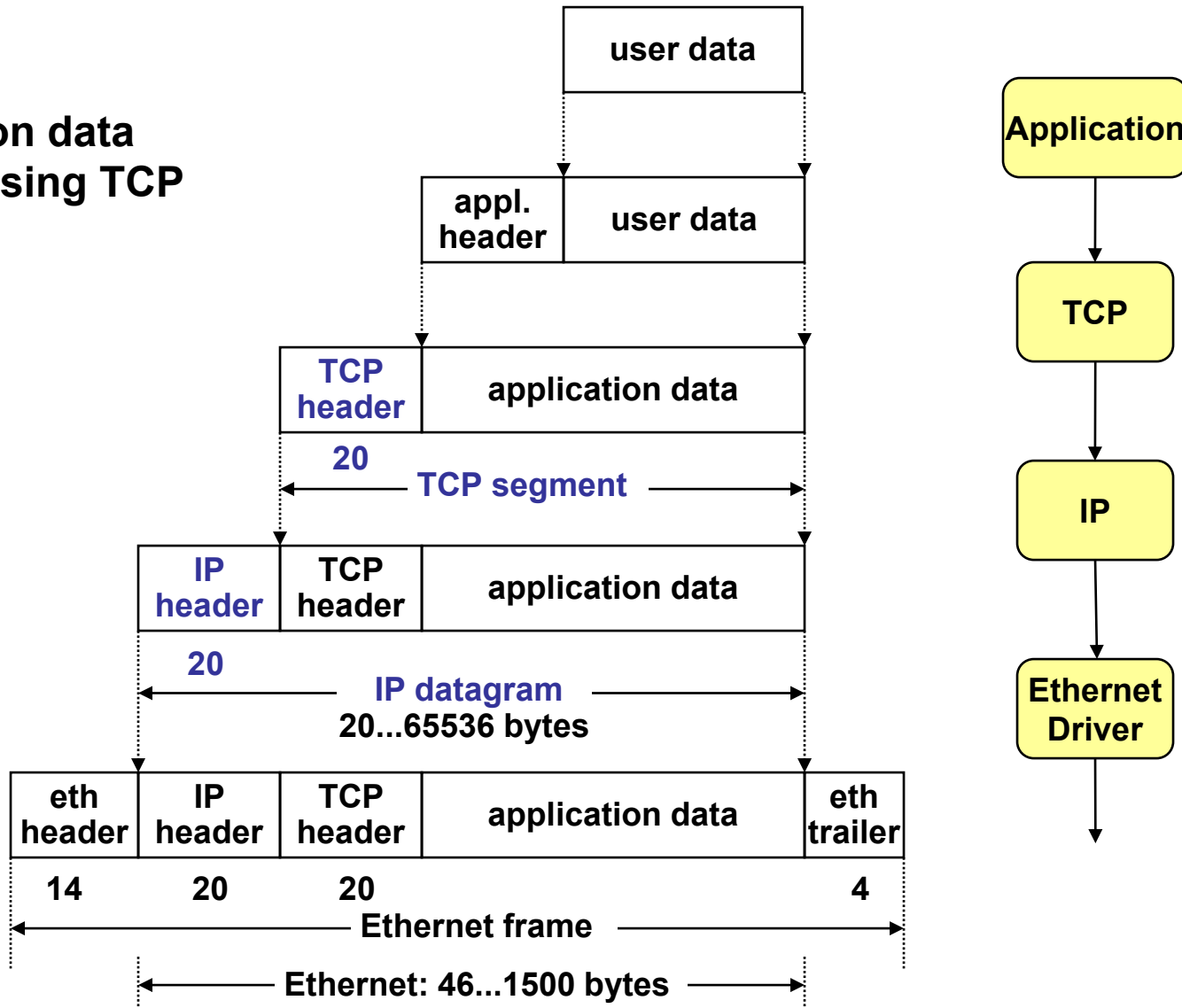
# TCP/IP Layer Overview

| TCP/IP Layers<br>(OSI model*) | Tasks                                                       | Protocol Examples                          |
|-------------------------------|-------------------------------------------------------------|--------------------------------------------|
| Application<br>(7)            | Application specific                                        | Telnet, rlogin, FTP, SMTP, SNMP, HTTP, ... |
| Transport<br>(4)              | End-to-end flow of data between application processes       | TCP, UDP                                   |
| Network<br>(3)                | Routing of packets between hosts                            | IP, ICMP                                   |
| Link<br>(2)                   | Hardware interface<br>Packet transfer between network nodes | PPP, Ethernet, IEEE 802.x, ARP             |

\* Mapping between TCP/IP and OSI layers is not always exact.

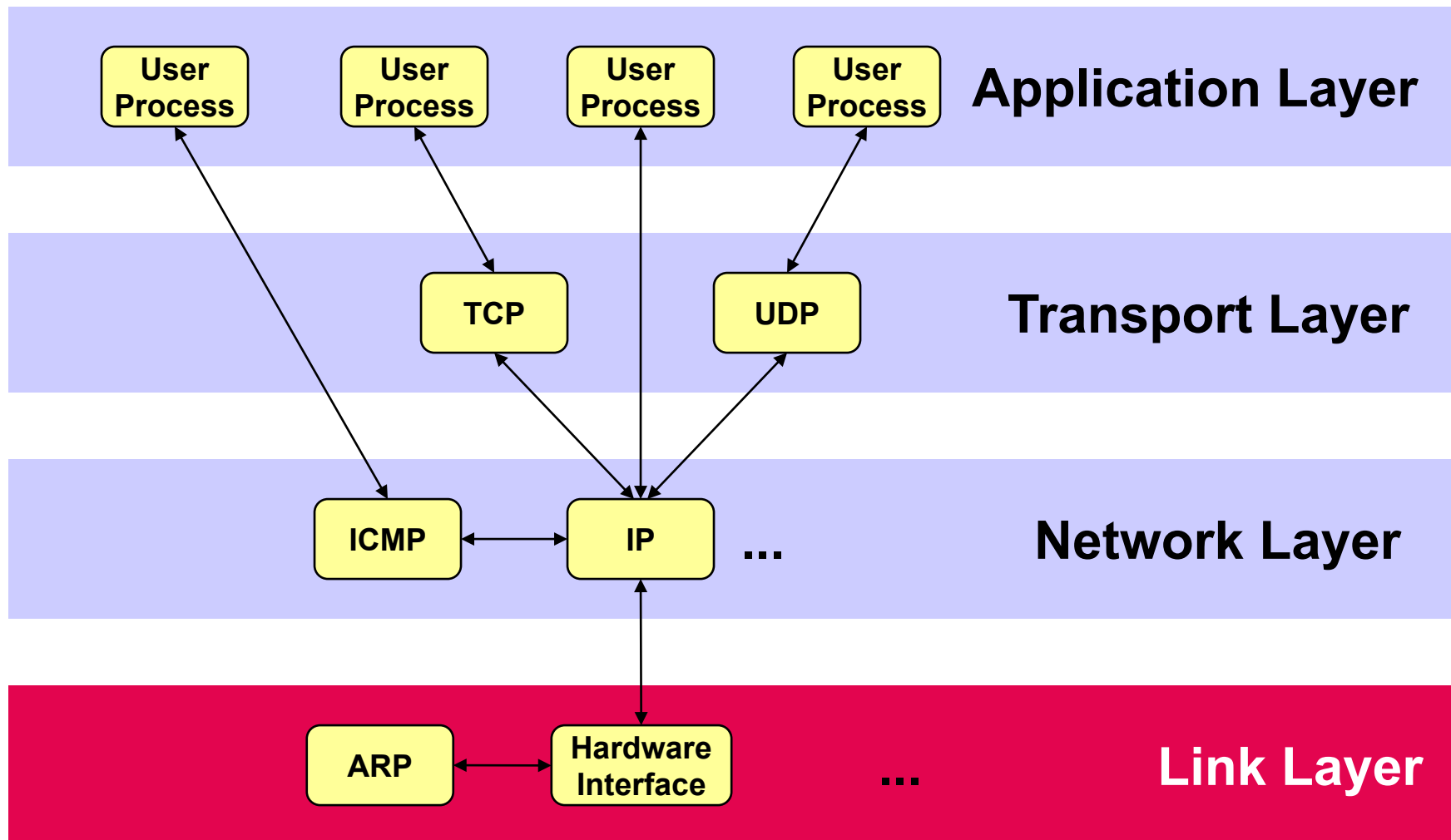
# TCP/IP Encapsulation

**Example:**  
**Application data**  
**transfer using TCP**





# TCP/IP Basics: Link Layer



# Link Layer Protocols

## Examples:

- **Ethernet** (encapsulation of higher layer packets is defined in RFC 894)
- **PPP**: Point-to-Point Protocol for serial lines (RFCs 1332, 1548)

## **MTU**: Maximum Transfer Unit (or Max. Transmission Unit)

- Maximum IP packet size in bytes (e.g. for Ethernet: 1500, X.25 Frame Relay: 576)

## Path MTU:

- Smallest MTU of any data link in the path between two hosts
- Used to avoid IP fragmentation
- TCP option: path MTU discovery (RFC 1191)

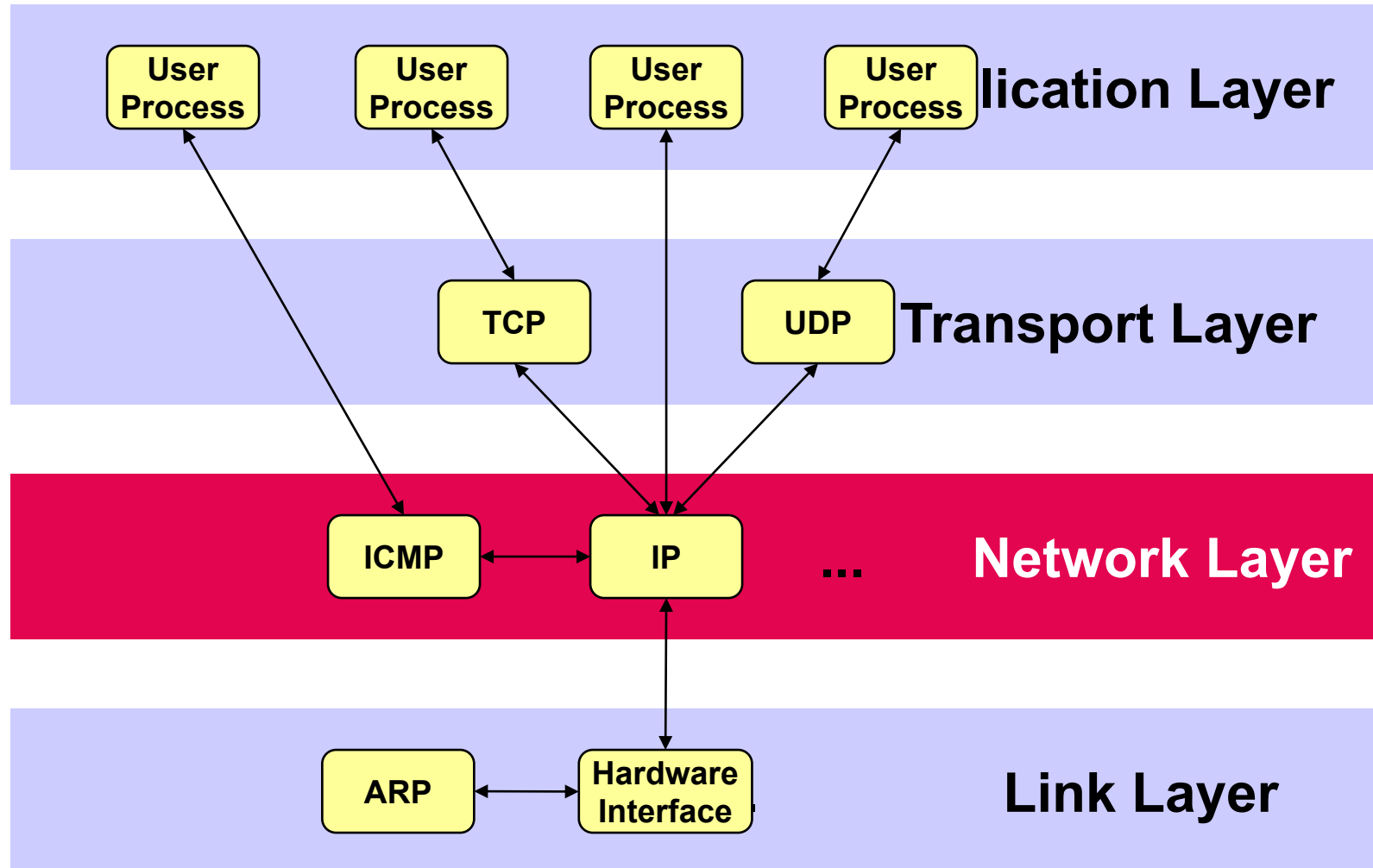
## Loopback Interface:

- A client application can connect to the corresponding server application *on the same host* by using the loopback IP address “**localhost**” = 127.0.0.1
- Implemented at the link layer, i.e. full processing of transport and IP layers

## **ARP**: Address Resolution Protocol (RFC 826)

- Address resolution from 32-bit IP addresses to hardware addresses (e.g. 48-bit)

# TCP/IP Basics: Network Layer



# IP: Internet Protocol

IP provides forwarding between hosts:

- **Based on 32-bit IP addresses** \*
- Hop-by-hop using routing tables

Unreliable, connectionless datagram delivery service:

- **packet loss, out-of-order delivery, duplication**

IP fragmentation: used on any link with MTU < original datagram length:

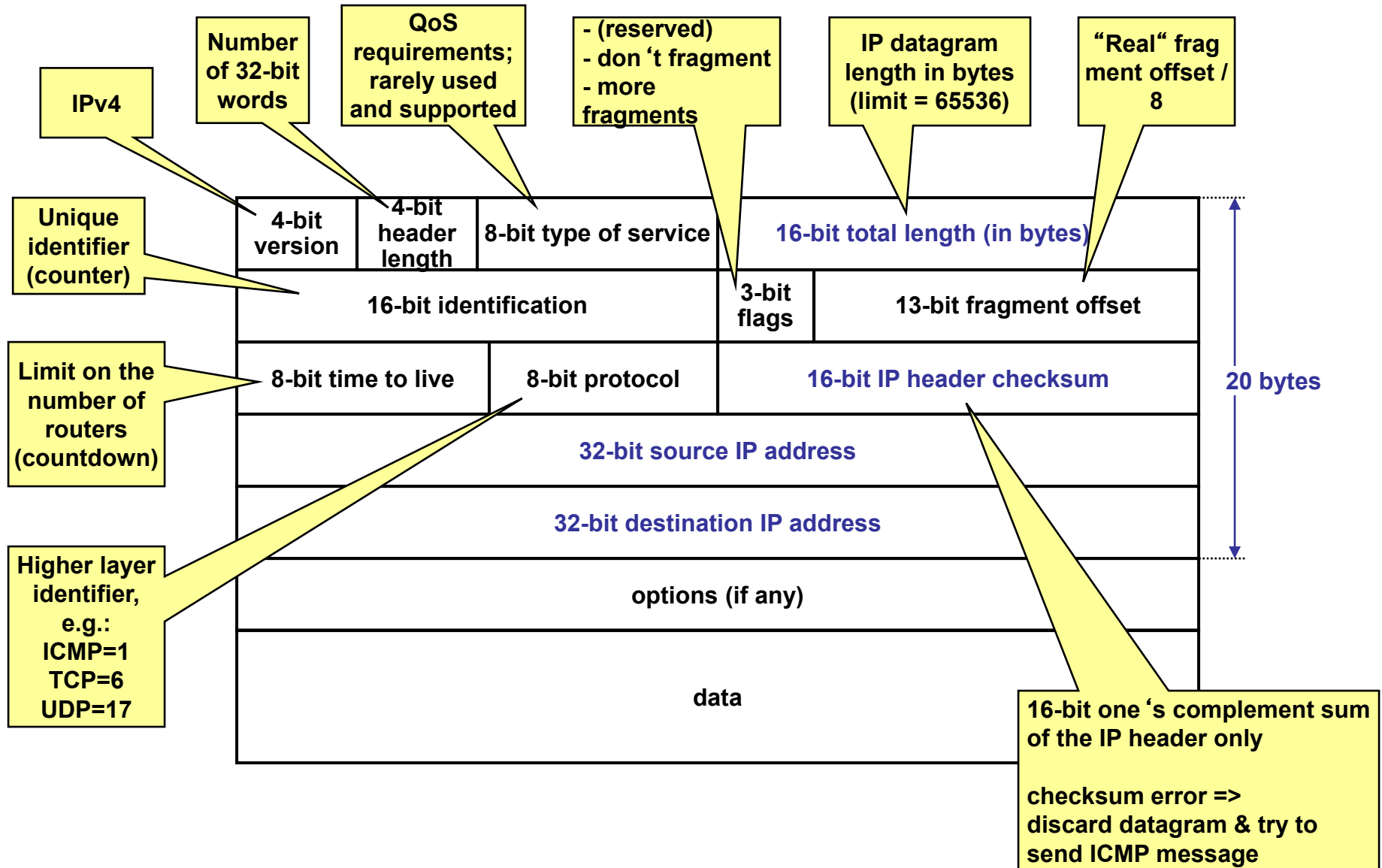
- Duplicates IP header for each fragment and sets flags for re-assembly
- Re-assembly at the receiving host only, never in the network

RFC 791

Applications use the Domain Name Service (**DNS**) to convert hostnames (e.g. “www.nokia.com”) into IP addresses (88.221.214.58) and vice-versa

\* IPv6 uses 128-bit addresses

# IP Datagram Format



# ICMP: Internet Control Message Protocol

ICMP packet consists of IP header + ICMP message

Used for queries and to communicate error messages back to the sender, e.g.:

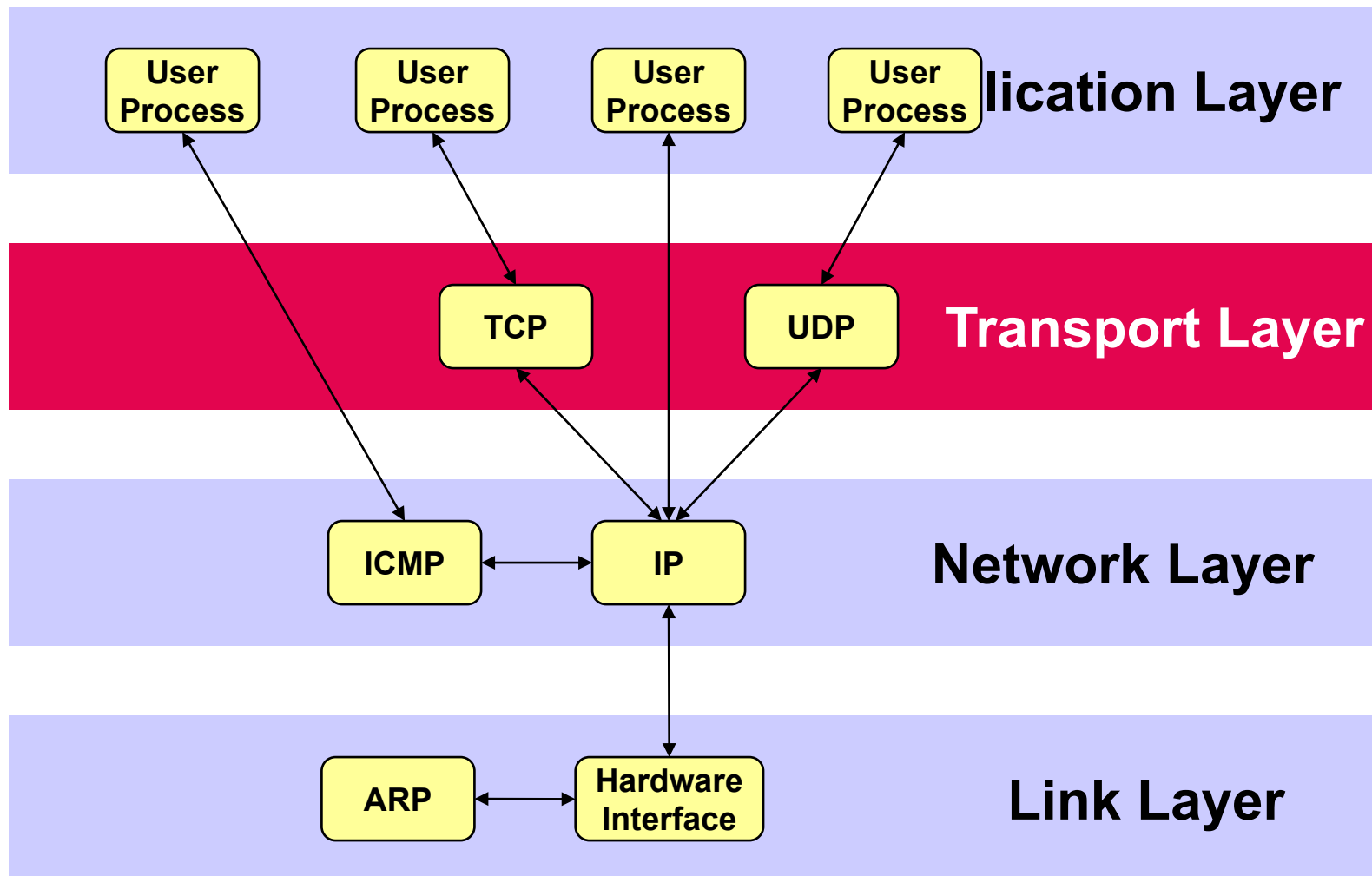
- “Bad IP header”
- “echo request” (or reply)
- “host unreachable”
- Mobile IP messages

Messages are used by higher layers, e.g.:

- ping, traceroute, TCP, ... HTTP

RFC 792

# TCP/IP Basics: Transport Layer



# UDP vs. TCP

**UDP:** User Datagram Protocol (RFC 768)

- Simple, unreliable, datagram-oriented transport of application data blocks

**TCP:** Transmission Control Protocol (RFC 793 + others)

- Connection-oriented, reliable byte stream service
- Details: see section on TCP

**Port numbers** are used for application multiplexing:

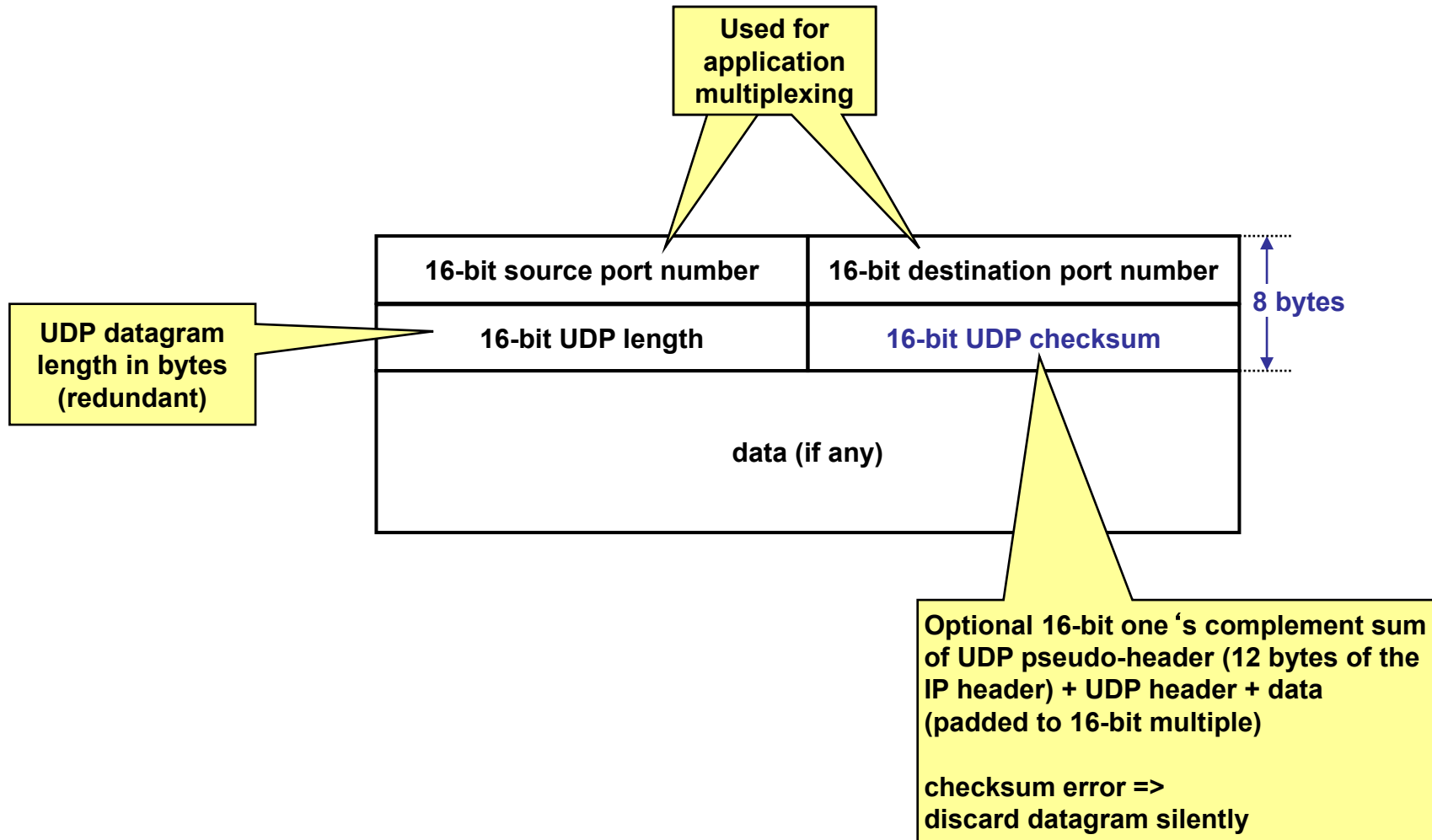
- Unique address = IP address + port number = “**socket**”
- Concept of well-known ports, e.g. TCP port 21 for FTP (RFC 1340)

Popular API for TCP and UDP connections: **Socket API**

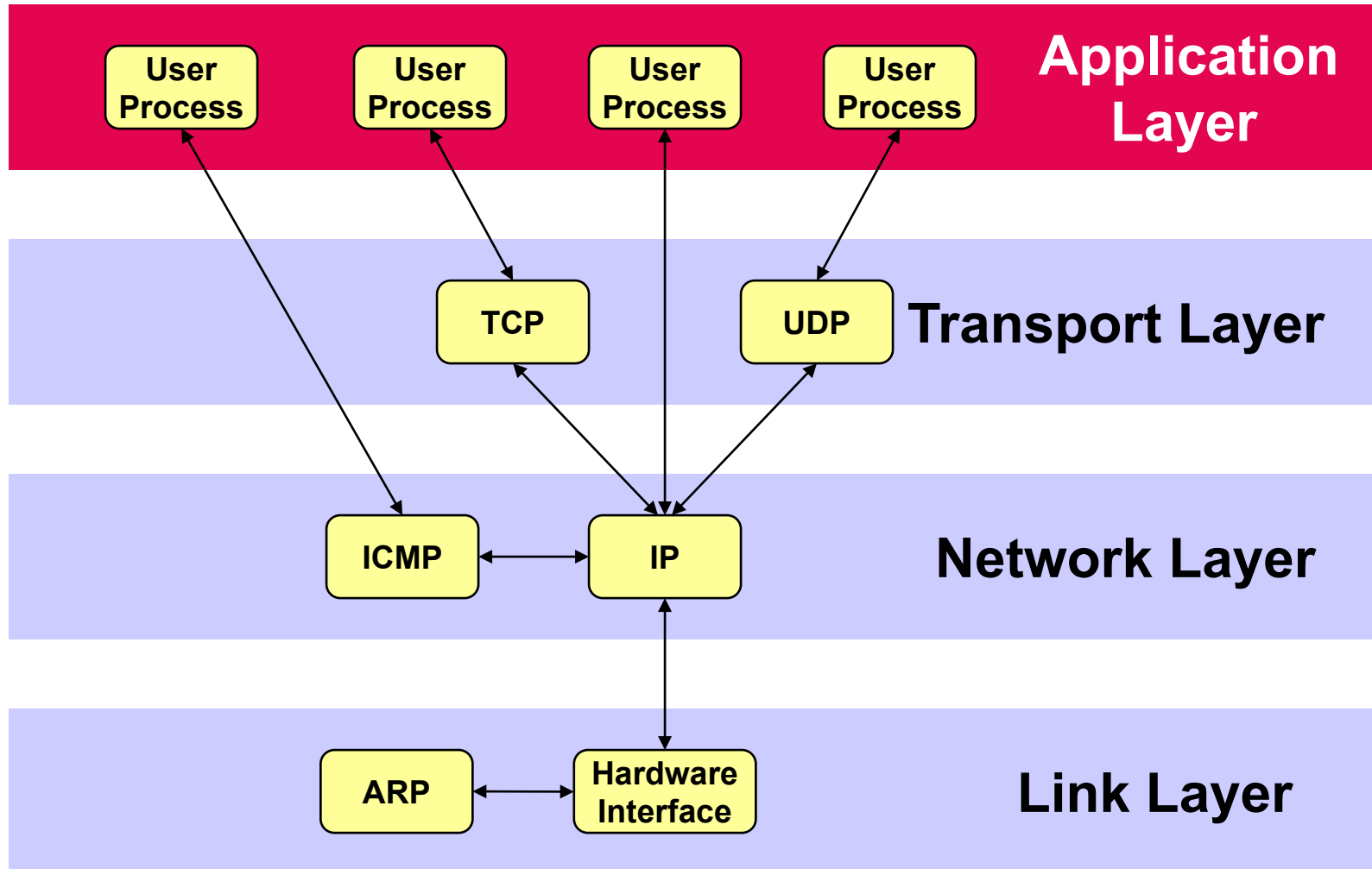
- “Stream sockets” use TCP
- “Datagram sockets” use UDP



# UDP Datagram Format



# TCP/IP Basics: Selected Applications



# FTP: File Transfer Protocol

File transfer based on **TCP**

TCP control connection:

- To well-known server port 21
- ASCII commands

TCP data connection

QoS requirements:

- High throughput (optimise TCP bulk data flow)

RFC 959

# Telnet and Rlogin

Used for remote login based on **TCP**

- Rlogin (RFC 1282):
  - Simple protocol designed for UNIX hosts
- Telnet (RFC 854):
  - Any OS
  - Option negotiation
  - More flexible and better performance

Client operation principle:

- Send each keystroke to the server
- Option: TCP's Nagle algorithm groups multiple bytes into one segment
- Display every response from the server

QoS requirements:

- Low-RTT transport of small packets (optimise TCP interactive data flow)
- $RTT = \text{round-trip-time (sender – receiver – sender)}$

# HTTP: Hypertext Transfer Protocol

Transfer of webpages based on **TCP**:

- Webpage typically consists of an HTML (Hyper Text Markup Language) document + various embedded objects, e.g. pictures

HTTP/1.0:

- Objects are (requested and received) **serially**
- For each object, a **new TCP connection** is established, used and released
- **Multiple connections**: several TCP connections can be used in parallel

# HTTP: Hypertext Transfer Protocol

HTTP/1.1 (RFC 2068): performance improvements by

- **Persistent Connections:**

- TCP connections are not released after each object, but used for the next one
  - avoids TCP connection establishment and termination
  - avoids slow start for each new connection

- **Pipelining:**

- Multiple objects can be requested in one packet
- Requested objects are sent sequentially over one TCP connection

HTTP/2 (RFC 7540): decreased latency due to

- Parallel loading of page elements over single TCP connection
- Header compression
- Server initiated data transmission (push technology)

# RTP: Real-time Transport Protocol

Transfer of real-time data based on **UDP**

RTP:

- for media with real-time characteristics (**audio/video**)
- services: payload type specification, sequence numbering, **timestamping**, source identification & synchronization, delivery monitoring
- no guaranteed quality of service (QoS)

RTCP (Real-time Transport Control Protocol):

- QoS monitoring & periodic feedback:
  - Sender report (synchronization, expected rates, distance)
  - Receiver report (loss ratios, jitter)

Network independent: on top of **unreliable, low-delay transport service**

RFC 1889

ITU-T H.225.0 Annex A => H.323 => e.g. MS Netmeeting, VoIP

# Summary: Internet Protocol Suite

The TCP/IP protocol suite is a heterogeneous family of protocols for the global Internet

At the center and always used: **IP**

- Routing between hosts

Application data transport by

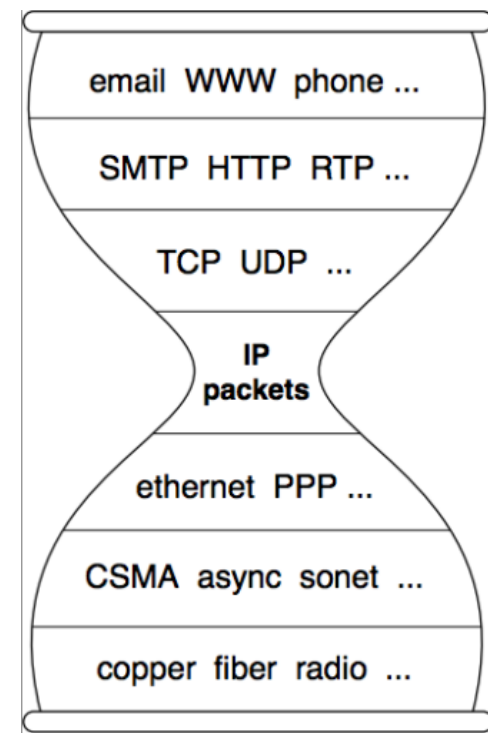
- **UDP**: unreliable datagram service
- **TCP**: reliable byte-stream service

TCP/IP stack is part of each operating system:

- Numerous **different implementations** and **bugs** exist

TCP performance is extremely important!

- TCP carries 62% of the flows, 85% of the packets, and **96% of the bytes** of Internet traffic (<http://www.cs.columbia.edu/~hgs/internet/traffic.html>)
- TCP's complex error control mechanism is **designed for wired networks**  
=> special problems for wireless transport





# Content

- **Internet Protocol Suite**
- **TCP**
  - TCP Datagram Format
  - Connection Setup and Release
  - MTU and MSS
  - Cumulative, Delayed and Duplicate Acknowledgements
  - Sliding Window Mechanism
  - Flow and Error Control
- **TCP over Wireless**
- **Beyond TCP**

# TCP (Transport Control Protocol)

Questions to be answered:

- How does the MSS differ from the MTU?
- What is the purpose of the MSS?
- What is the purpose of the different flags?
- What is the purpose of the checksum field?
- What is the purpose of the window size field?
- What is the purpose of DUPACKs?
- How does a TCP sender learn about overload in the network? Which messages and mechanisms are used for this?
- How does the TCP congestion control algorithm in the sender react to overload problems? What are the options?
- Why does TCPs response to timeouts differ from the response for DUPACKs? What is the purpose of this?
- Can we find out the direction in which the congestion occurs?

# TCP (Transmission Control Protocol)

## Properties

### Connection-oriented, reliable byte-stream service:

- Reliability by ARQ (Automatic Repeat reQuest):
  - TCP receiver sends acknowledgements (acks) back to TCP sender to confirm delivery of received data
  - Cumulative, positive acks for all contiguously received data
  - Timeout-based retransmission of segments
- TCP transfers a byte stream:
  - Segmentation into TCP segments, based on MTU
  - Header contains byte sequence numbers

### Congestion avoidance + flow control mechanism

*In the following examples:*

- *Packet sequence numbers (instead of byte sequence numbers)*
- ***ack i** acknowledges receipt of packets through packet i (instead of bytes)*

# TCP Segment Format

**TCP is full duplex:**

**Each segment contains an ack for the reverse link**

**A "pure" ack is a segment with empty data**

Identifies the number of the first data byte in this segment within the byte stream

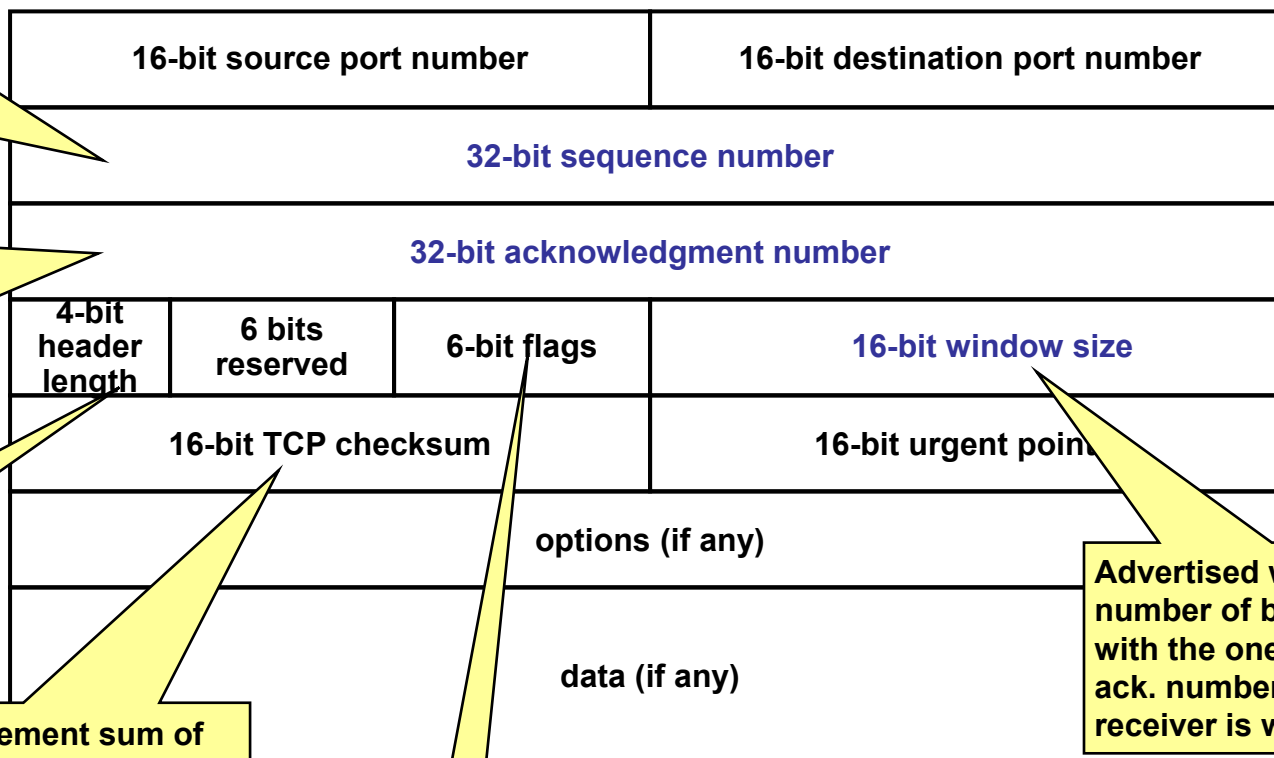
Ack for the reverse link: next sequence number that is expected to be received

Number of 32-bit words

16-bit one's complement sum of TCP pseudo-header (12 bytes of the IP header) + TCP header + data (padded to 16-bit multiple)

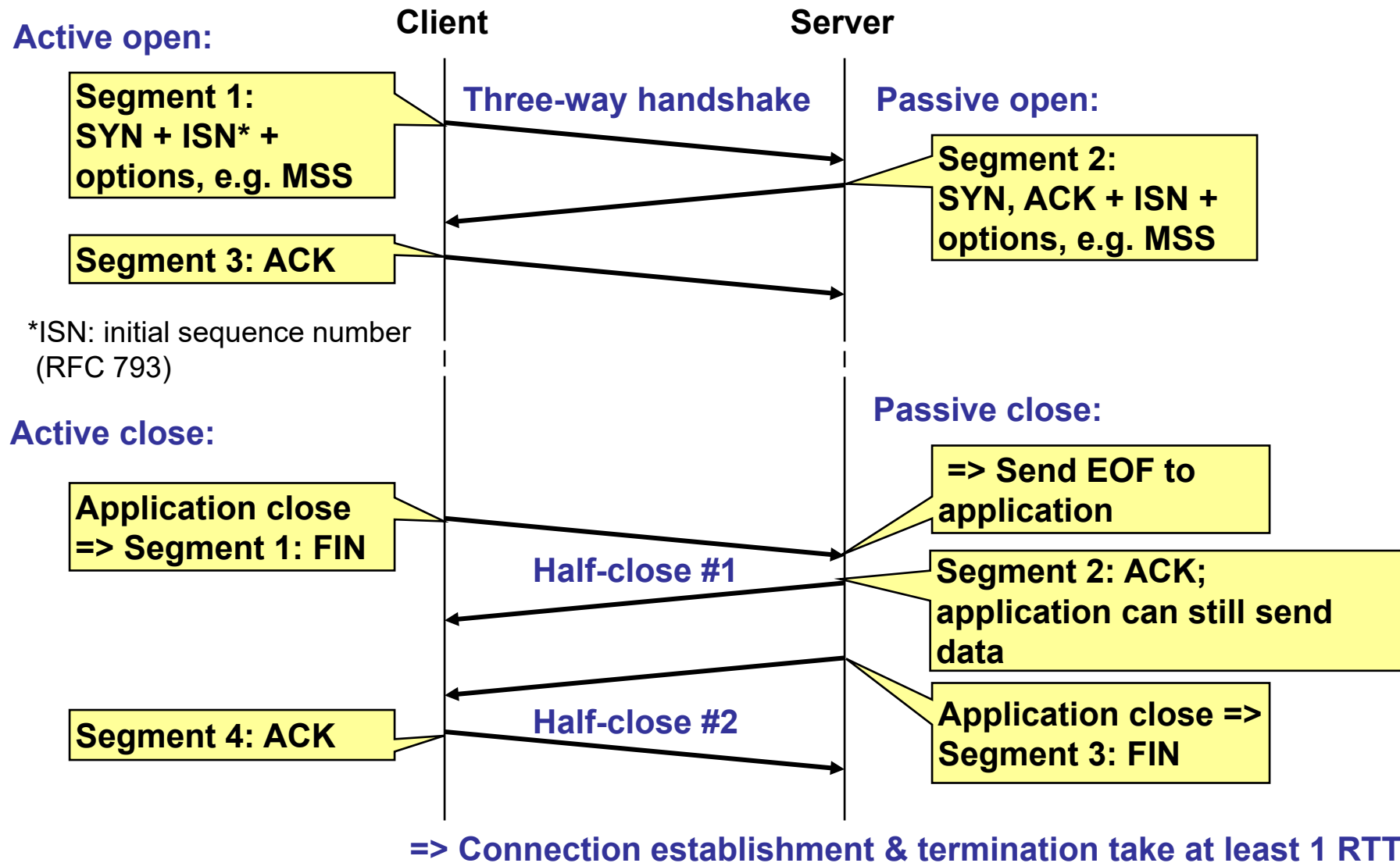
checksum error  
=> discard datagram silently!  
=> using an erroneous header is dangerous; loss will be detected by other mechanisms

**URG:** Urgent Pointer field significant - urgent data are outstanding  
**ACK:** Acknowledgment field significant  
**PSH:** Push Function - push to indicate prompt transmission of data  
**RST:** Reset the connection  
**SYN:** Synchronize sequence numbers  
**FIN:** No more data from sender

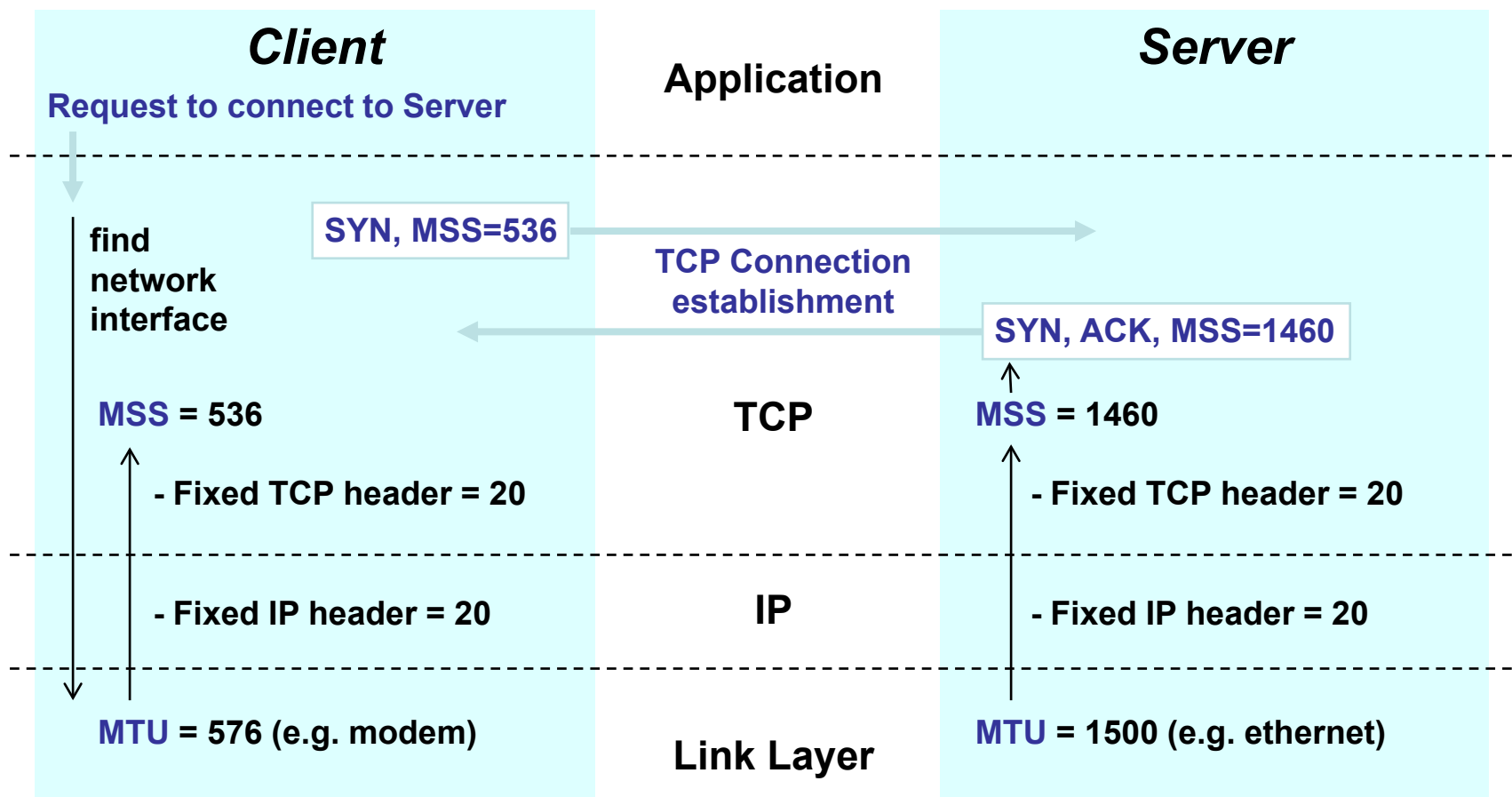


Advertised window size: number of bytes, starting with the one specified in the ack. number field, that the receiver is willing to accept

# TCP Connection Establishment and Termination



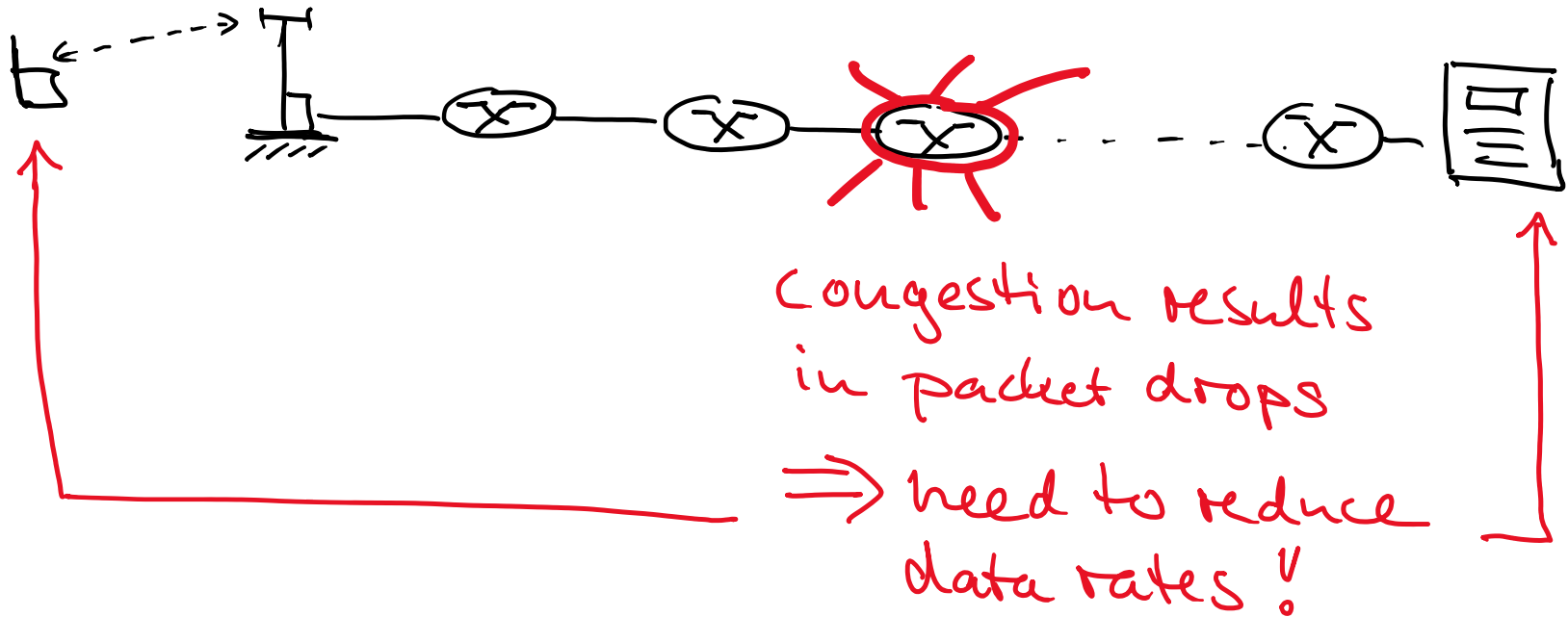
# MTU and MSS: Maximum Segment Size



MSS is optionally announced (**not negotiated**) by each host at TCP connection establishment. The smaller value is used by both ends, i.e. 536 in the above example. Note that “real” TCP payload is smaller if TCP options are used.

# TCP congestion control

The Problem:

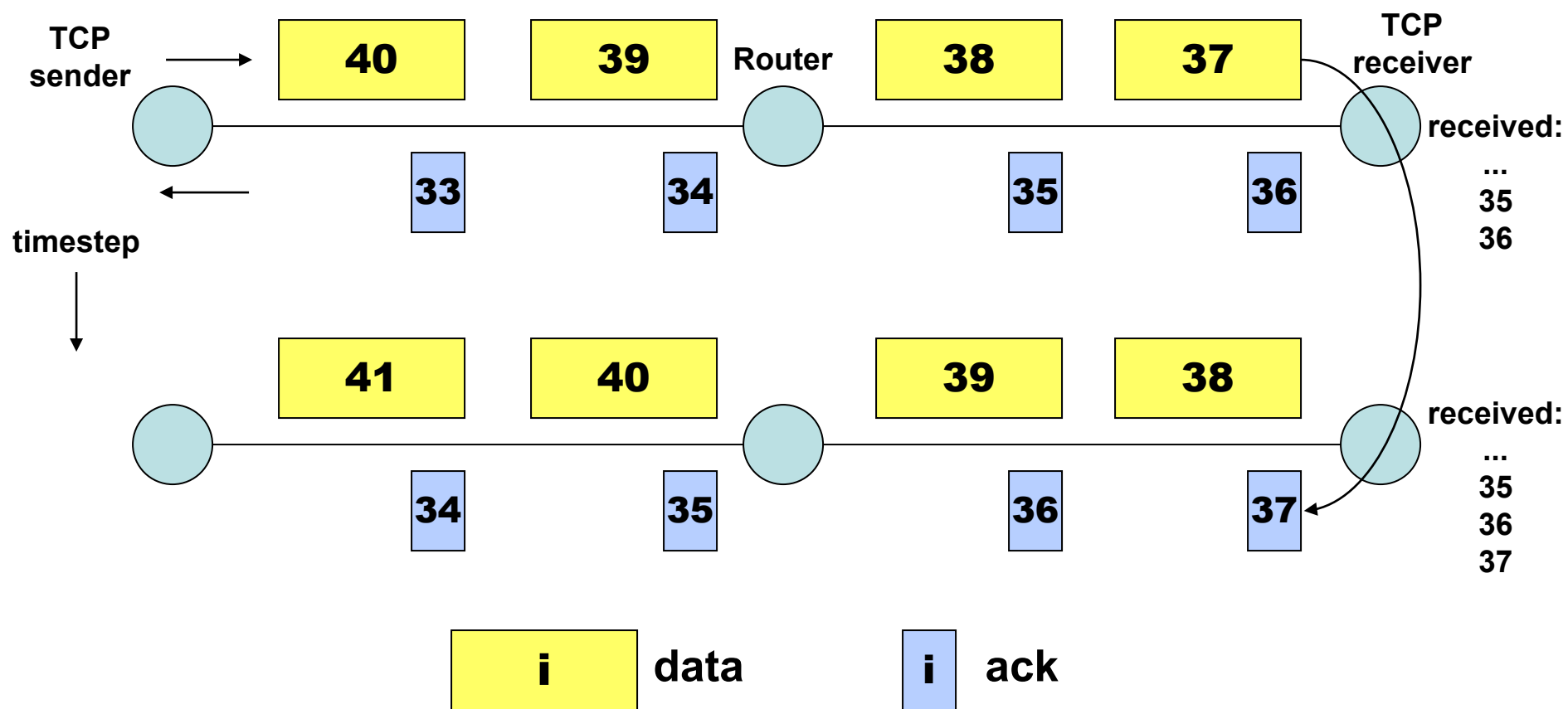


This is an end-to-end issue

Globally: distributed control issue with many players

# Cumulative Acknowledgements

A new cumulative ack is generated only on receipt of a **new in-sequence** segment





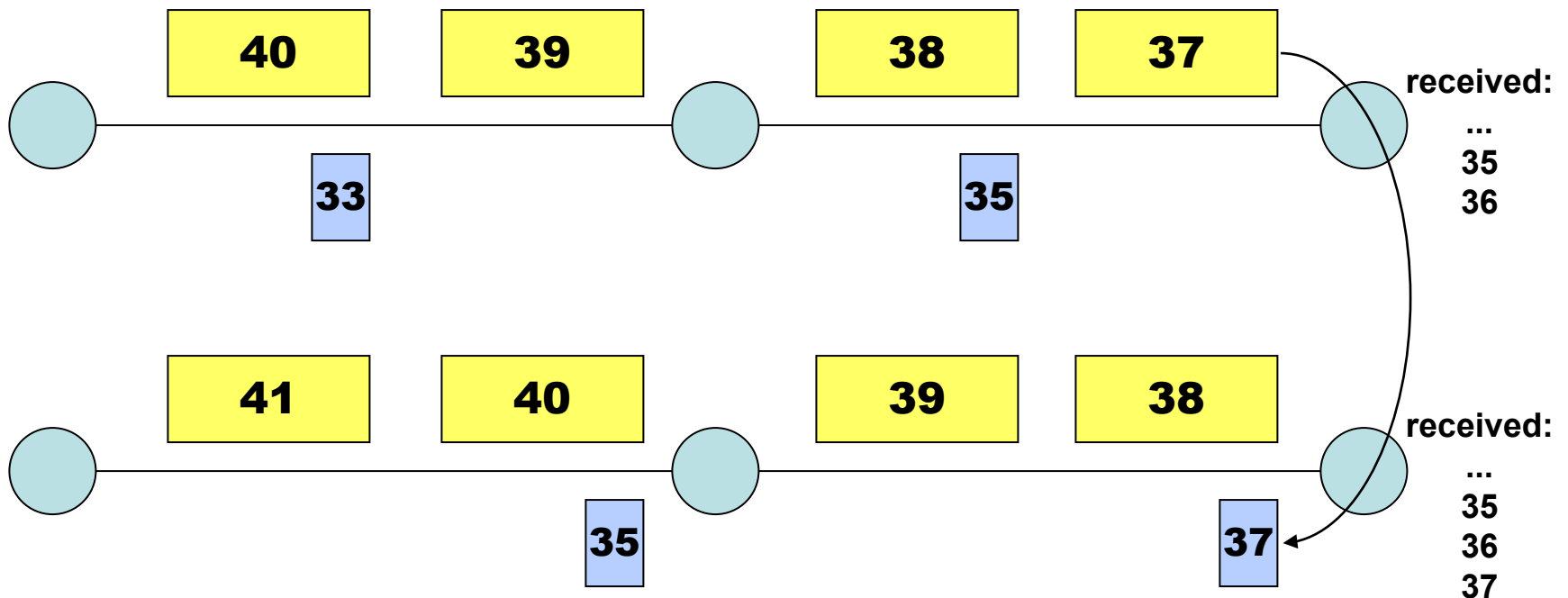
# Delayed Acknowledgements

Delaying acks reduces ack traffic

An ack is delayed until

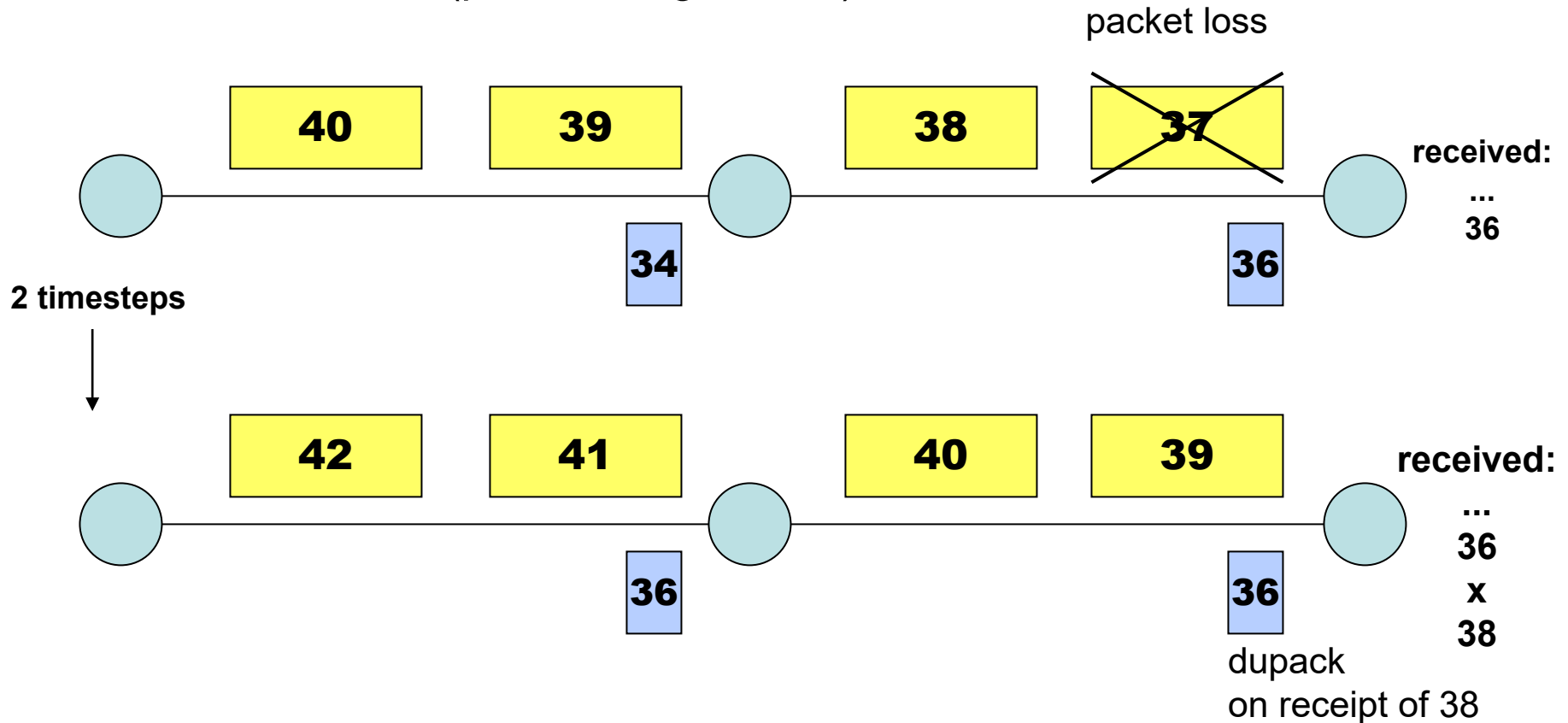
- another segment is received, or
- delayed ack timer expires (200 ms typical)

New ack not produced on receipt of segment 36, but on receipt of 37



# Duplicate Acknowledgements 1

A **dupack** is generated whenever an **out-of-order** segment arrives at the receiver (packet 37 gets lost)

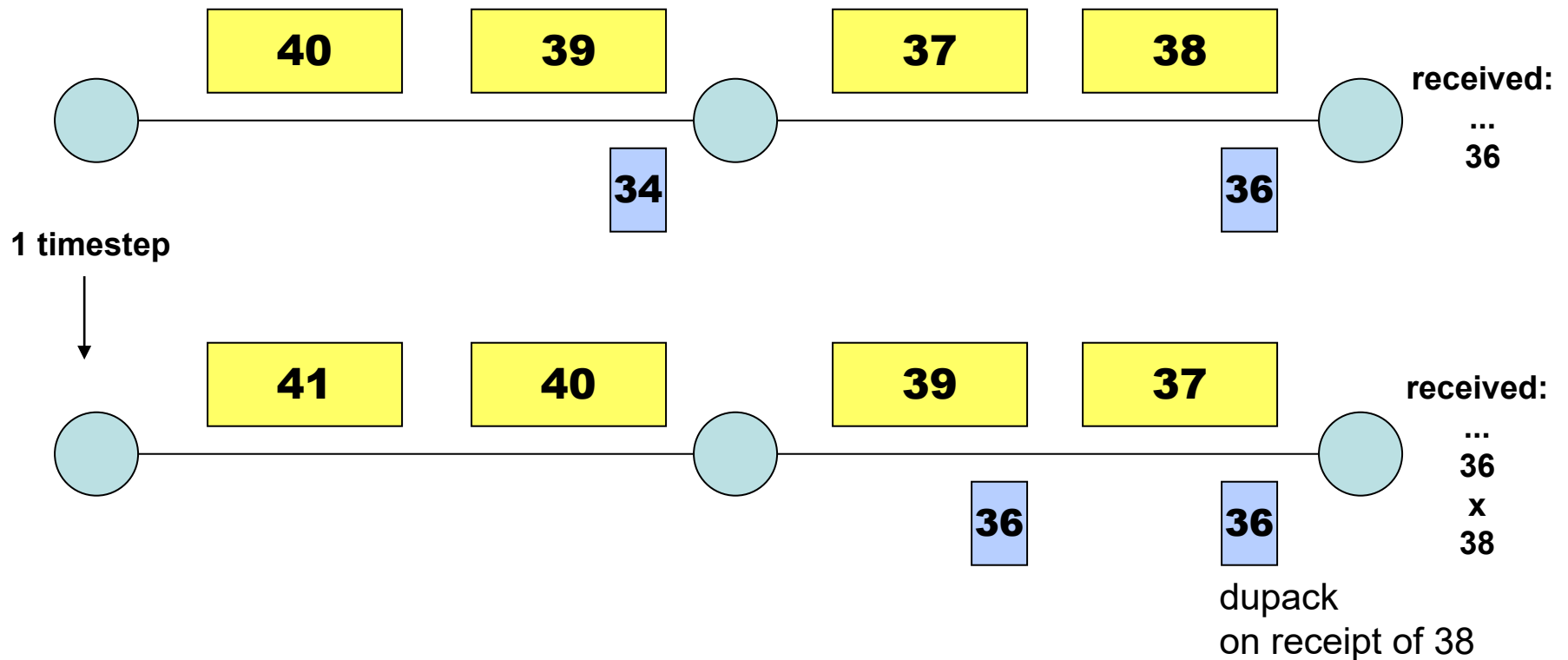


## Duplicate Acknowledgements 2

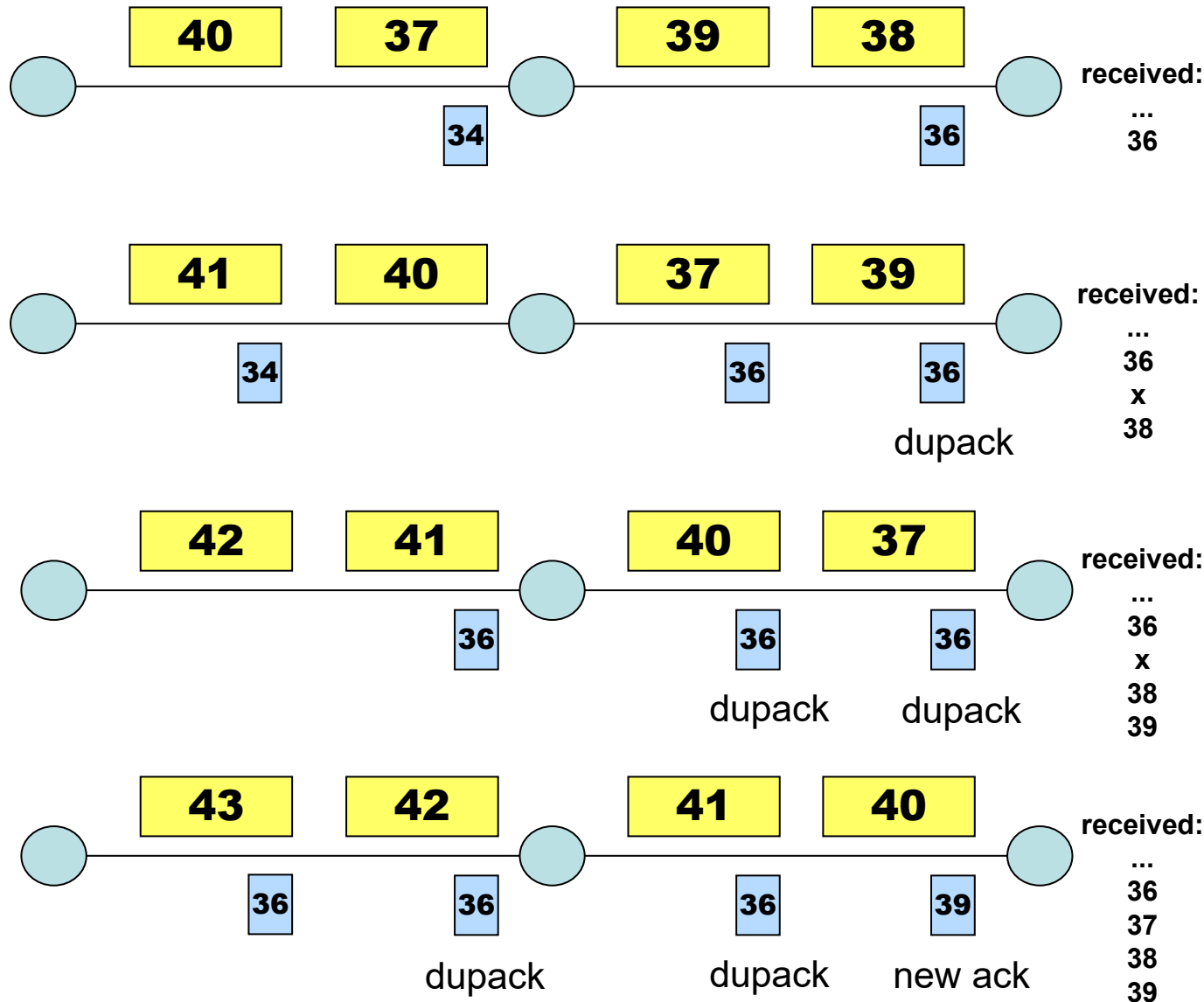
# Dupacks are not delayed

## Dupacks may be generated when

- a segment is **lost** (see previous slide), or
- a segment is delivered **out-of-order**:



# Duplicate Acknowledgements 3

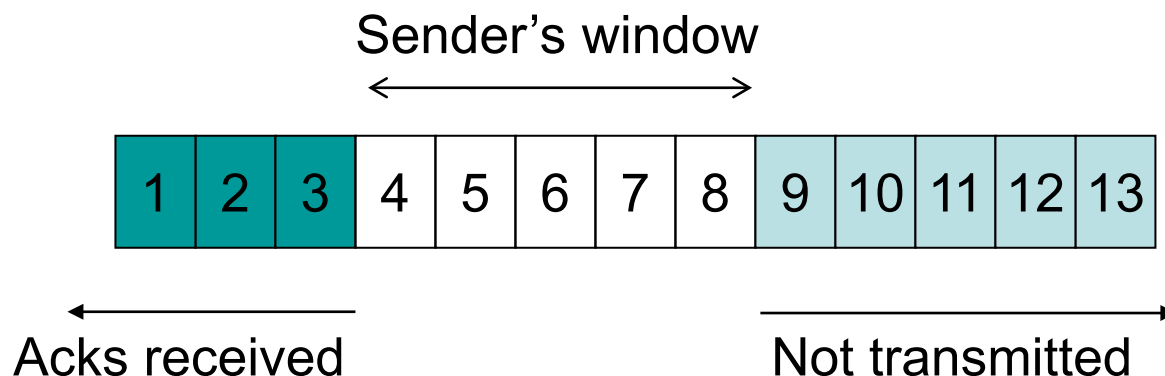


Number of dupacks depends on how much out-of-order a packet is

A series of dupacks allows the sender to guess that a single packet has been lost

# Window Based Flow Control 1

## Sliding window protocol



Window size  $W$  is minimum of

- **receiver's advertised window** - determined by available buffer space at the receiver and signaled with each ack
- **congestion window** - determined by the sender, based on received acks

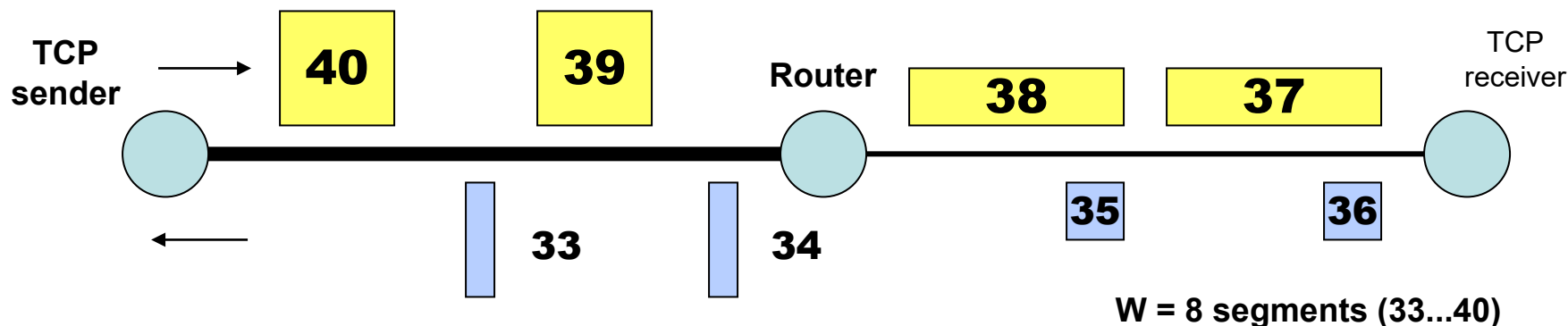
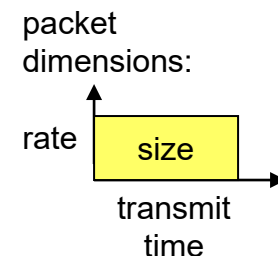
TCP's window based flow control is “**self-clocking**”:

- New segments are sent when outstanding segments are ack'd

# Window Based Flow Control 2

Optimum window size:

- $W = \text{data rate} * \text{RTT} = \text{“bandwidth-delay product”}$   
(optimum use of link capacity: “pipe is full”)



What if window size is too large?

- ❑ Queuing at intermediate routers (e.g. at wireless access point)  
=> increased RTT due to queuing delays  
=> potential of packet loss

What if window size is too small?

- ❑ Inefficiency: unused link capacity

# Packet Loss Detection Based on Timeout

TCP sender starts a timer for a segment (only one segment at a time)

If ack for the timed segment is not received before timer expires,  
outstanding data are assumed to be lost and retransmitted

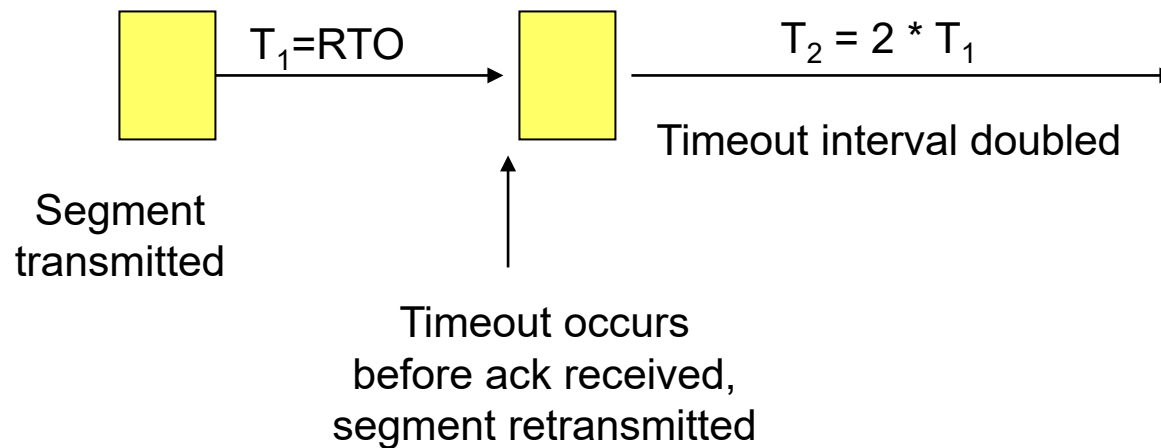
=> **go-back-N ARQ**

**Retransmission timeout (RTO)** is calculated dynamically based on  
measured RTT:

- $RTO = \text{mean RTT} + 4 * \text{mean deviation of RTT}$ 
  - Mean deviation  $\delta = \text{average of } |\text{sample} - \text{mean}|$  is easier to calculate than standard deviation (and larger, i.e. more conservative)
- Large variations in the RTT increase the deviation, leading to larger RTO
- RTT is measured as a discrete variable, in multiples of a “tick”:
  - 1 tick = 500 ms in many implementations
  - smaller tick sizes in more recent implementations
- RTO is at least 2 clock ticks

# Exponential Backoff

Double RTO on successive timeouts:



Total time until TCP gives up is up to 9 min

Rationale: allow an intermediate, congested router to recover

Problem: if ack is lost, TCP (sender) just waits for the next timeout



# Packet Loss Detection Based on Dupacks: Fast Retransmit Mechanism

TCP sender considers timeout as a strong indication that there is a severe link problem

On the other hand, continuous reception of dupacks indicates that following segments are delivered, and the link is ok

=> TCP sender assumes that a (single) packet loss has occurred if it receives **three dupacks consecutively**

=> Only the (single) missing segment is retransmitted

=> **selective-repeat ARQ**

Note: 3 dupacks are also generated if a segment is delivered at least 3 places out-of-order

=> Fast retransmit useful only if lower layers deliver packets **“almost ordered”** - otherwise, unnecessary fast retransmit

# Flow Control by the Sender

## Slow Start

Initially, congestion window size ( $cwnd$ ) = 1 MSS

Increment  $cwnd$  by 1 MSS on each new ack

Slow start phase ends when  $cwnd$  reaches  $ssthresh$  (slow-start threshold)

=>  $cwnd$  grows exponentially with time during slow start (in theory)

- Factor of 1.5 per RTT if every other segment is ack'd
- Factor of 2 per RTT if every segment is ack'd
- In practice: increase is slower because of network delays (see next slide)

## Congestion Avoidance

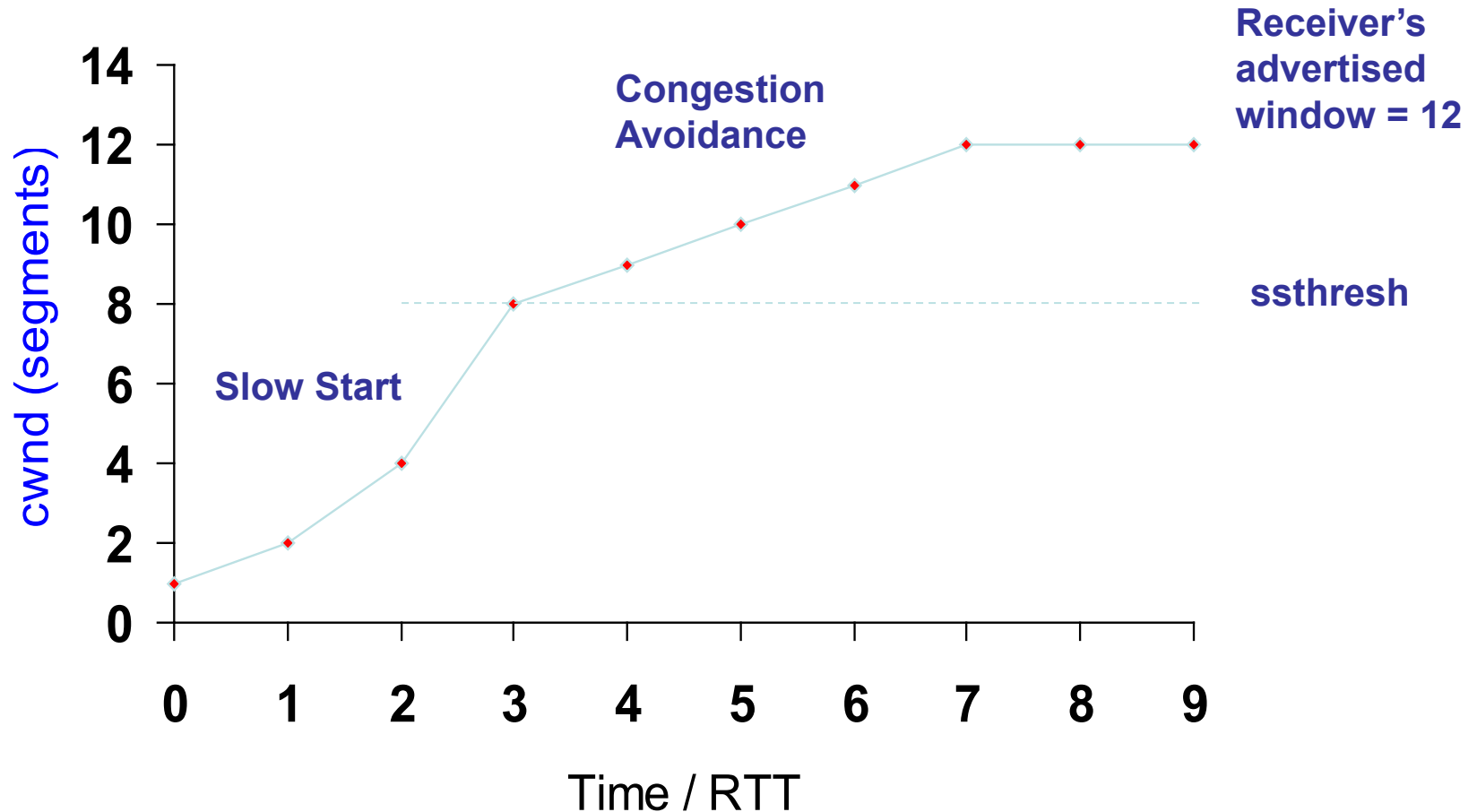
On each new ack, increase  $cwnd$  by  $1/cwnd$  segments

(here  $cwnd$  denotes number of packets of size MSS; for  $cwnd$  in bytes:  $cwnd = cwnd + MSS^2/cwnd$ )

=>  $cwnd$  grows linearly with time during congestion avoidance (in theory)

- 1/2 MSS per RTT if every other segment ack'd
- 1 MSS per RTT if every segment ack'd

# Slow Start & Congestion Avoidance – Theory



- Theoretical assumption: after sending  $n$  segments,  $n$  acks arrive within one RTT
- Note that Slow Start **starts slowly, but speeds up quickly**

# Slow Start – Reality (Including Network Delay)

- Taking network delay into account, “cwnd increases exponentially” turns into:
    - **cwnd increases sub-exponentially**
    - **pairs of segments are sent** while pipe fills
  - Simple example:
    - one-way delay = 1 timestep
    - data rate = 1 segment/timestep
- sending rate > data rate (cwnd > 2)  
 (timestep 4 onwards)  
 => at some point in time there will be a **packet loss**, causing TCP to slow down

1 RTT {

| Time-step | Sender action         | cwnd | #segments sent | #segments outstanding |  | #segments rec'd and ack'd | Receiver action           |
|-----------|-----------------------|------|----------------|-----------------------|--|---------------------------|---------------------------|
| 0         | initial values        | 1    |                | 0                     |  |                           |                           |
|           | send segment 1        |      | 1              | 1                     |  |                           |                           |
| 1         |                       |      |                |                       |  | 1                         | receive and ack segment 1 |
| 2         | receive ack 1         | 2    |                | 0                     |  |                           |                           |
|           | send segments 2 and 3 |      | 2              | 2                     |  |                           |                           |
| 3         |                       |      |                |                       |  | 1                         | receive and ack segment 2 |
| 4         | receive ack 2         | 3    |                | 1                     |  | 1                         | receive and ack segment 3 |
|           | send segments 4 and 5 |      | 2              | 3                     |  |                           |                           |
| 5         | receive ack 3         | 4    |                | 2                     |  | 1                         | receive and ack segment 4 |
|           | send segments 6 and 7 |      | 2              | 4                     |  |                           |                           |
| 6         | receive ack 4         | 5    |                | 3                     |  | 1                         | receive and ack segment 5 |
|           | send segments 8 and 9 |      | 2              | 5                     |  |                           |                           |

←  
Buffer starts filling up

# Congestion Control after Packet Loss

**Packet loss detected by timeout** ( $\Rightarrow$  severe link problem):

Retransmit lost segments

Go back to **Slow Start**:

- Reduce **cwnd** to initial value of **1 MSS**
- Set **ssthresh** to half of window size before packet loss:
  - $\text{ssthresh} = \max((\min(\text{cwnd}, \text{receiver's advertised window})/2), 2 \text{ MSS})$

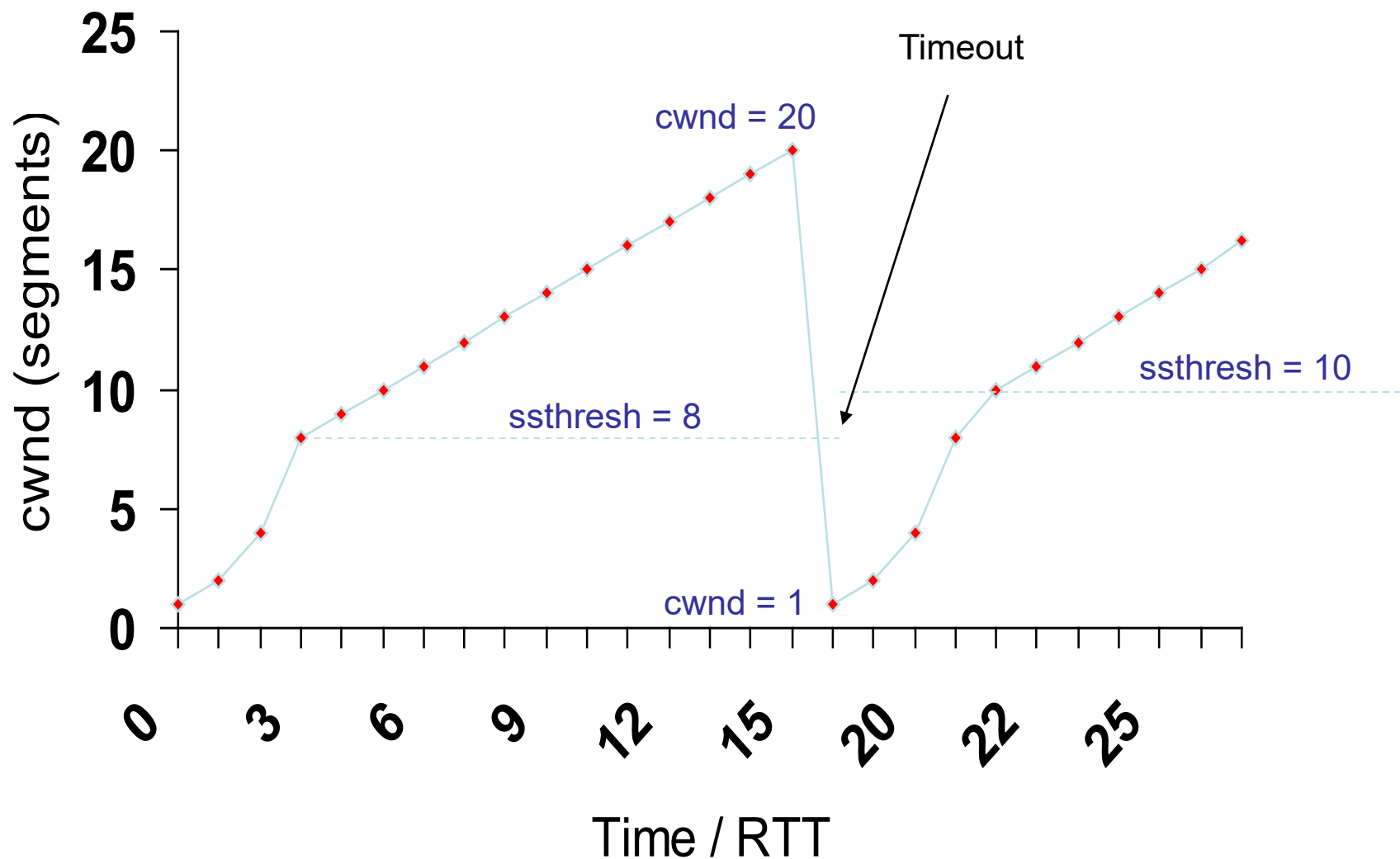
**Packet loss detected by  $\geq 3$  dupacks** ( $\Rightarrow$  single packet loss, but link is ok):

**Fast Retransmit** single missing segment

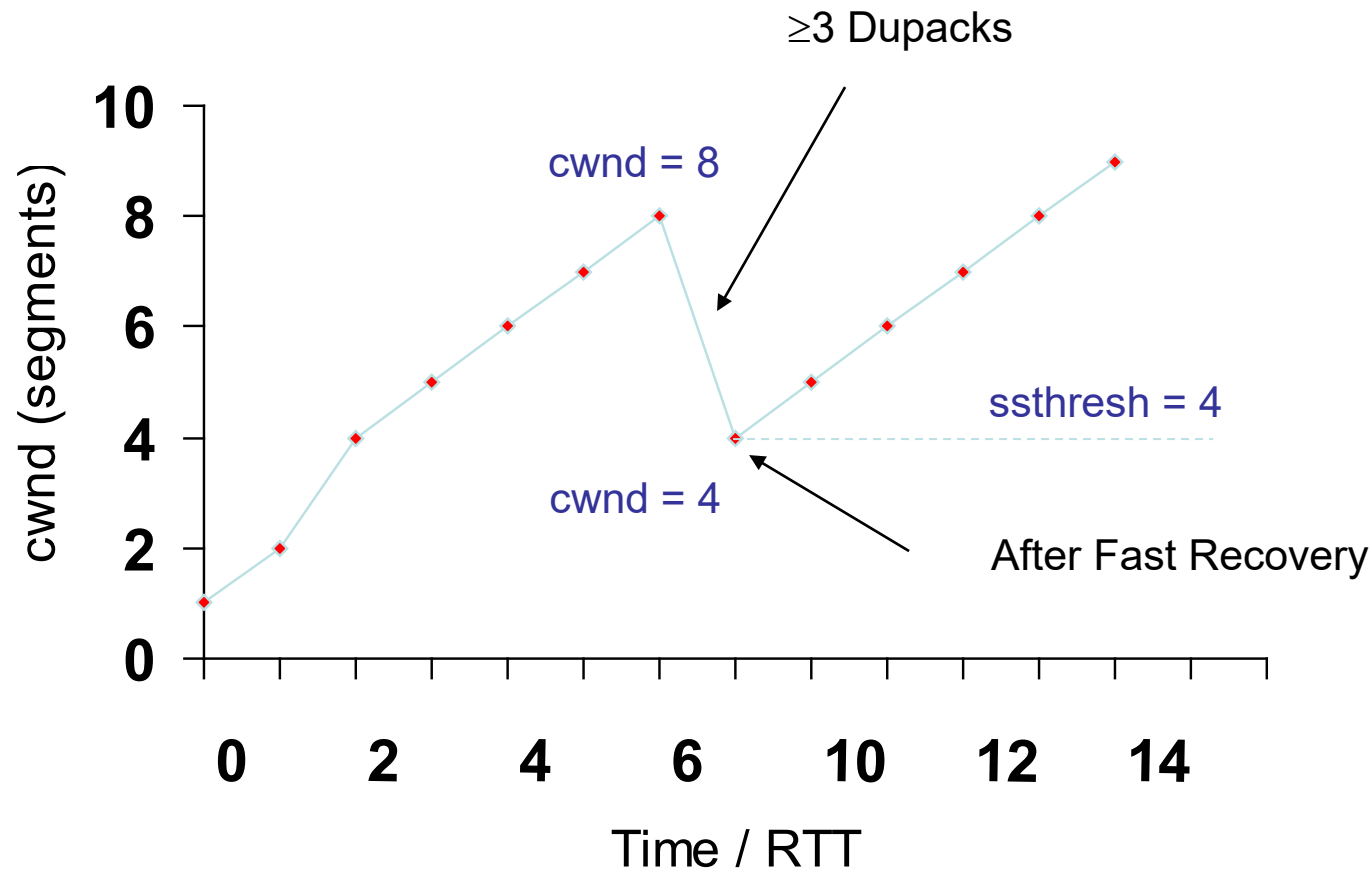
Initiate **Fast Recovery**:

- Set **ssthresh** and **cwnd** to half of window size before packet loss:
  - $\text{ssthresh} = \max((\min(\text{cwnd}, \text{receiver's advertised window})/2), 2 \text{ MSS})$
  - $\text{cwnd} = \text{ssthresh} + \text{number of dupacks}$
- When a new ack arrives: continue with **Congestion Avoidance**:
  - $\text{cwnd} = \text{ssthresh}$

# Packet Loss Detected by Timeout



# Packet Loss Detected by $\geq 3$ Dupacks



- After fast retransmit and fast recovery window size is reduced in half
- Multiple packet losses within one RTT can result in timeout

# Summary: TCP



TCP provides a **connection-oriented, reliable byte-stream service**:

- application data stream is transferred in **segments** based on lower layer **MTU**
- receiver sends back **cumulative acknowledgements** (acks)
- **sliding window** mechanism with **flow control** based on
  - receiver's **advertised window**,
  - sender's **Slow Start** (timeout) and **Congestion Avoidance** (3 DUPACKs) mechanisms
- Error control & packet loss detection based on
  - **adaptive retransmission timeout** => back to **Slow Start**,
  - **duplicate acknowledgments** (dupacks) => **Fast Retransmit & Fast Recovery**



# Content

- **Internet Protocol Suite**
- **TCP**
- **TCP over Wireless**
  - Problems of TCP in wireless environments
  - Improved TCP variants
  - Beyond TCP
- **Beyond TCP**

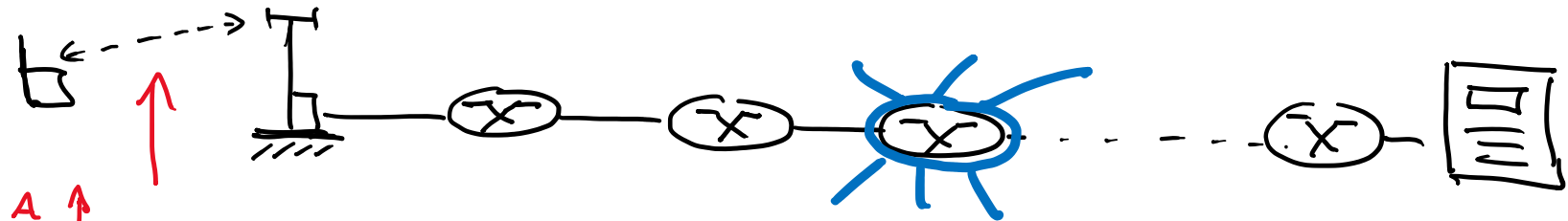
# TCP over wireless

Questions to be answered:

- What are the problems of the use of TCP over wireless links?
- Identify possible effects of high fluctuations of the channel capacity on TCP?
- What is the difference in the impact of temporarily poor radio channels for the case that LLC is empty or uses some ARQ over the wireless link?
- Can you think of mechanisms to alleviate the problem?
- What is the advantage of splitting TCP connections in a wired and a wireless part, what's the disadvantage?
- Are there less intrusive solutions to the TCP-over-wireless problem?
- What benefit can TCP draw from lower layer information? How can it be used?

# TCP congestion control for wireless

The Problem:



congestion results  
in packet drops

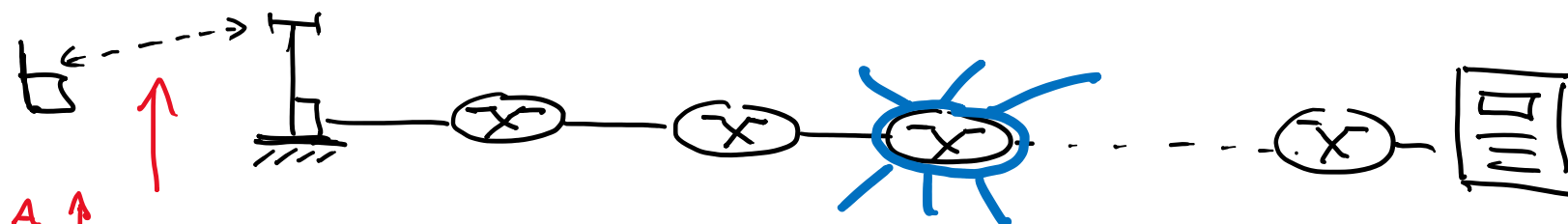
⇒ need to reduce  
data rates !

↓↓  
 packet drops  
 ↓↓  
 LLC empty or ARQ  
 ↓↓  
 TCP: DUPACKs or Timeout  
 ↓  
 decreased data rate

But: this is the wrong  
decision !

# TCP congestion control for wireless

The Problem:



congestion results  
in packet drops

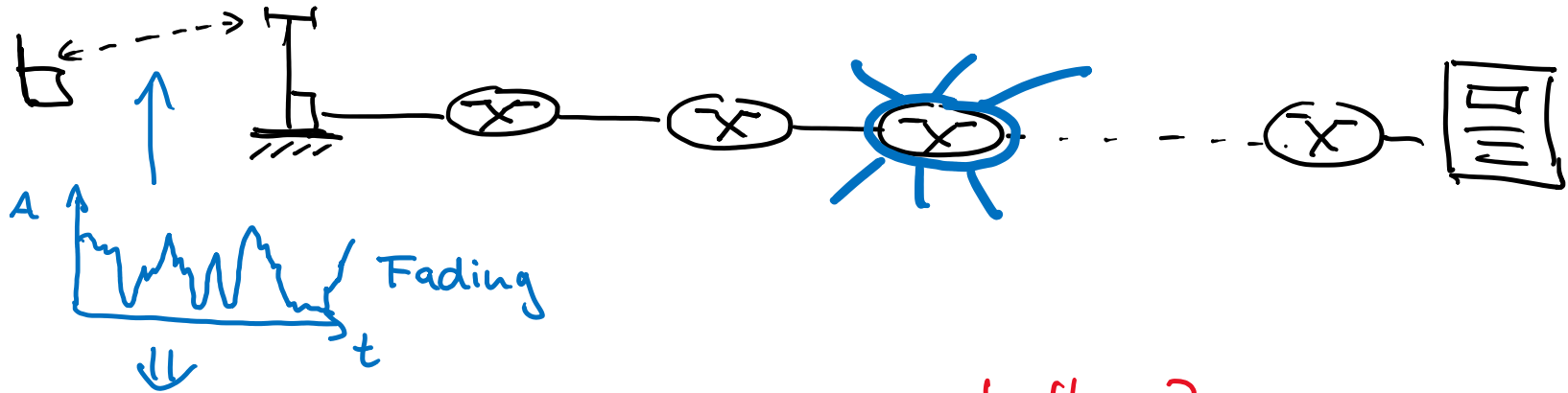
⇒ need to reduce  
data rates !

↓↓  
 increased delay due to suspended  
 transmission  
 ↓↓  
 TCP: Timeout  
 ↓↓  
 dramatic decrease of data rate

Again: this is the wrong  
decision !

# TCP congestion control for wireless

The Problem:



in any case  
TCP reduces data rate

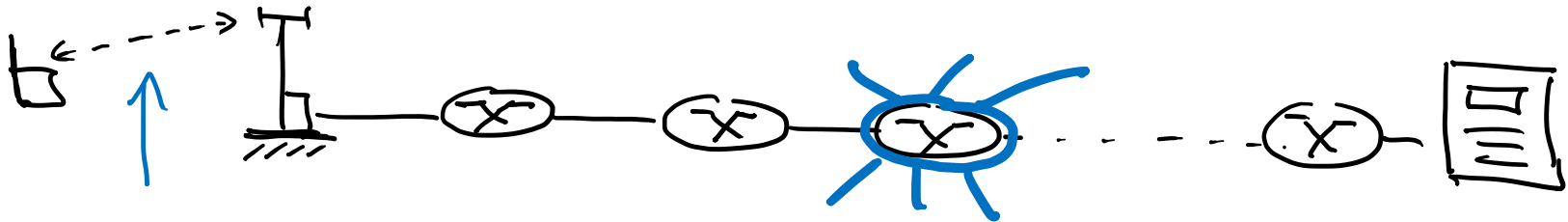
But: this is the wrong  
decision !

Why?

- packet errors do not imply overload on the wireless link
- there is no buffer overflow, at least for this case

# TCP congestion control for wireless

The Problem:



But there could  
be overload as  
well



TCP: need to reduce  
data rates

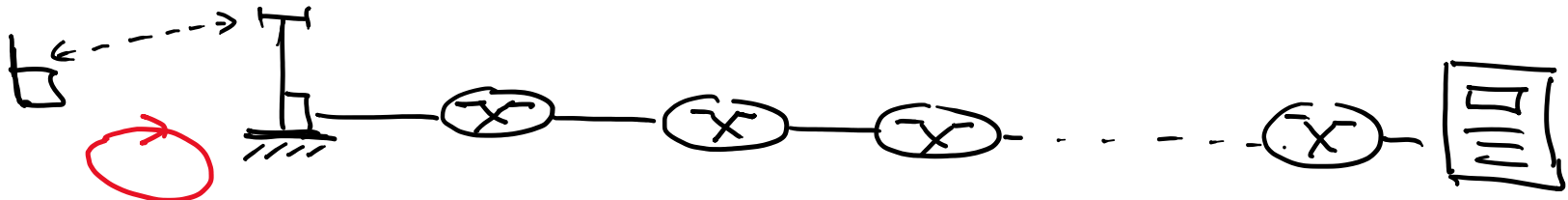
This time the  
decision is right!

So, how can we  
draw the correct  
conclusions from  
packet drops?

- slow data rate  
down
- or not?

# TCP congestion control for wireless

One solution: Local retransmit by LLC protocol



Retransmit  
locally on  
layer 2



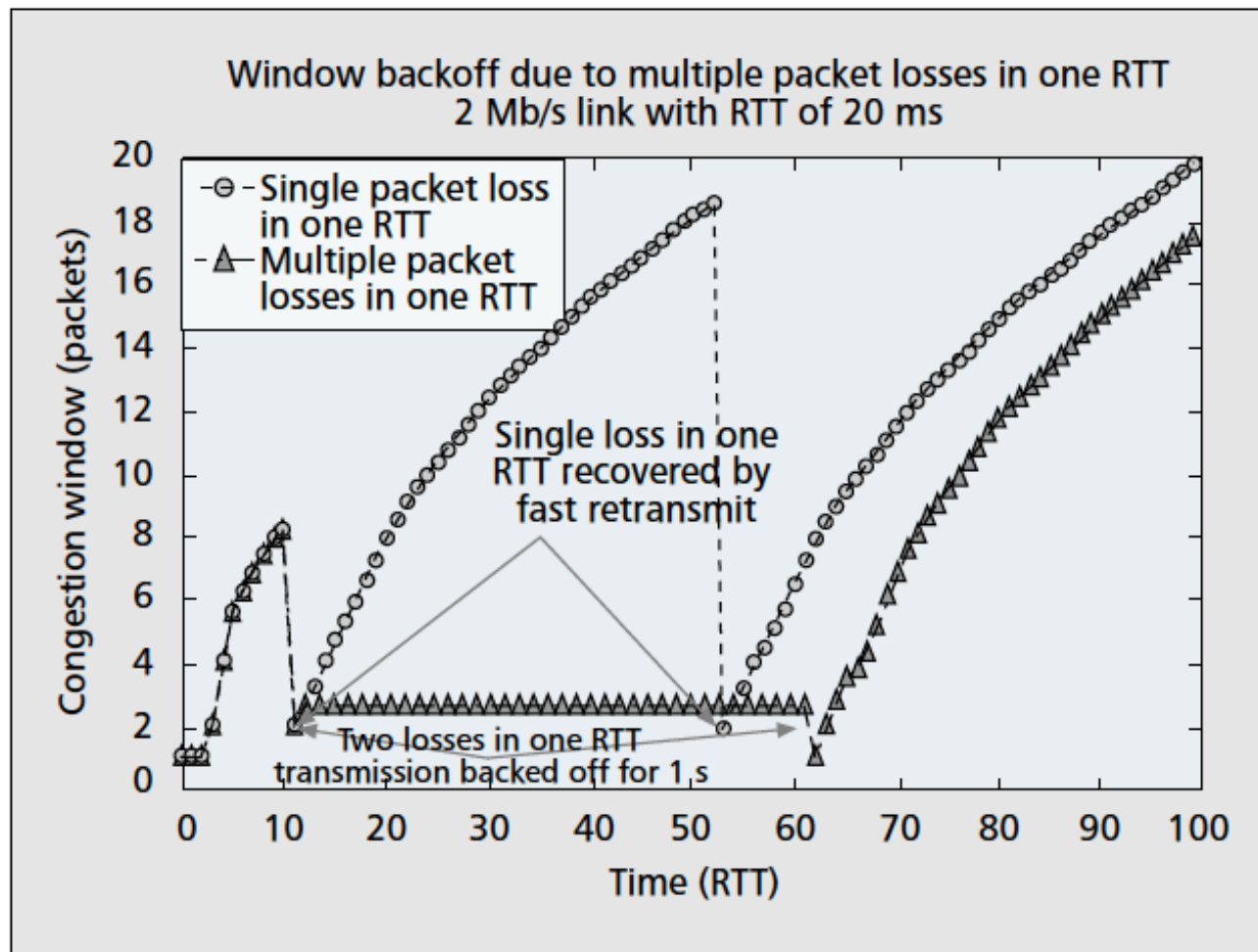
TCP will see a very  
high packet delay  
for some packets  
and out-of-order  
delivery of packets

The impact on the  
throughput highly depends  
on the TCP parameters



⇒ This may result in  
- retransmissions or  
- timeouts  
⇒ Drop of packet rate!

# Effects of single and multiple losses in one RTT



Source: Tian, Xu, Ansari: "TCP in Wireless Environments: Problems and Solutions",  
IEEE Communications Magazine, 43(3), March 2005

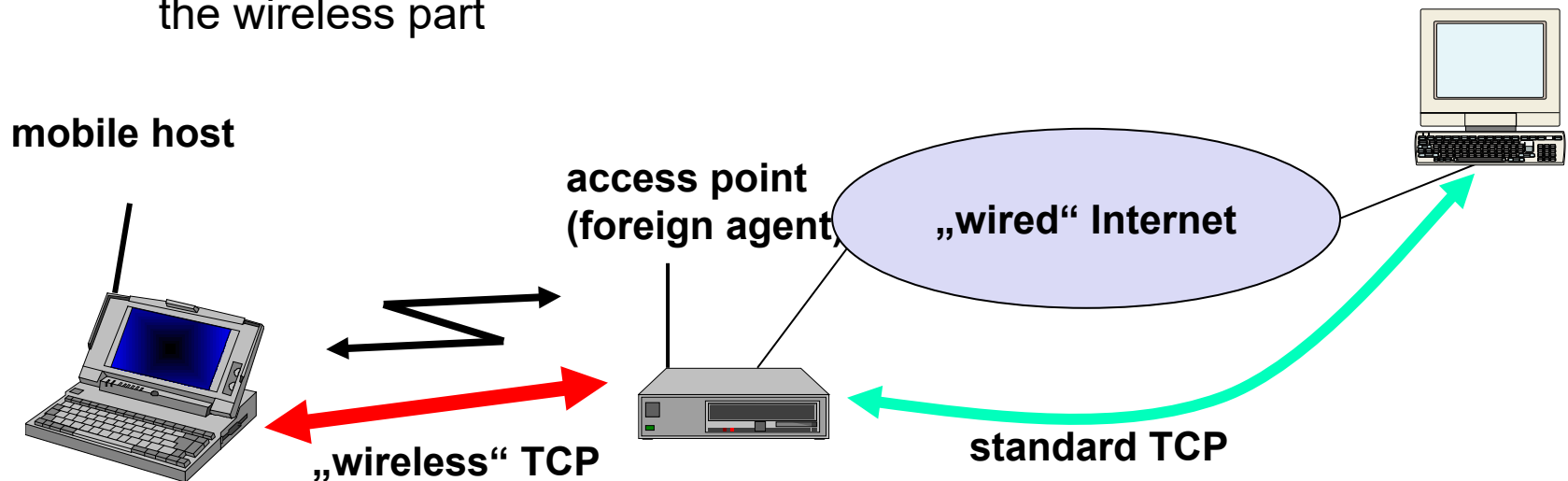


# TCP congestion control for wireless – problem summary

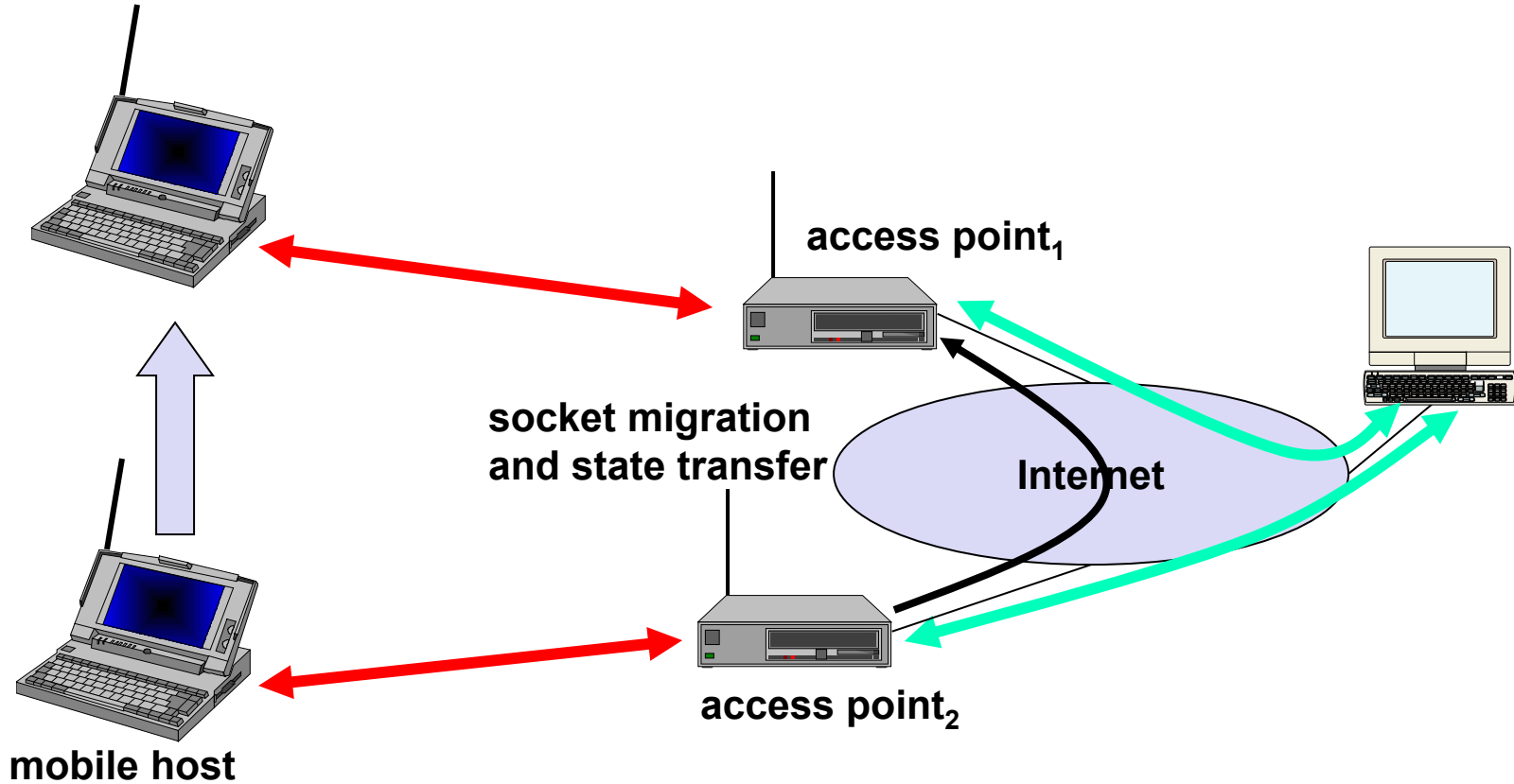
- TCP assumes congestion if packets are dropped
  - typically wrong in wireless networks, here we often have packet loss due to *transmission errors*
  - furthermore, *mobility* itself can cause packet loss, if e.g. a mobile node roams from one access point (e.g. foreign agent in Mobile IP) to another while there are still packets in transit to the wrong access point and forwarding is not possible
- TCP's self-clocking slow start mechanism depends on RTT
  - High latency of wireless systems prolongs the slow start period, which is especially crucial with short TCP connections (HTTP)
- The performance of an unchanged TCP degrades severely
  - however, TCP cannot be changed fundamentally due to the large base of installations in the fixed network
  - TCP on server does not know whether peers are mobile or not
  - the basic TCP mechanisms keep the whole Internet together

# Indirect TCP – Principle

- Indirect TCP (I-TCP) segments the connection
  - no changes to the TCP protocol for hosts connected to the wired Internet, millions of computers use (variants of) this protocol
  - optimized TCP protocol for mobile hosts
  - splitting of the TCP connection at, e.g., the foreign agent into 2 TCP connections, **no real end-to-end connection any longer**
  - hosts in the fixed part of the net do not notice the characteristics of the wireless part



# Indirect TCP – Socket and state migration due to handover



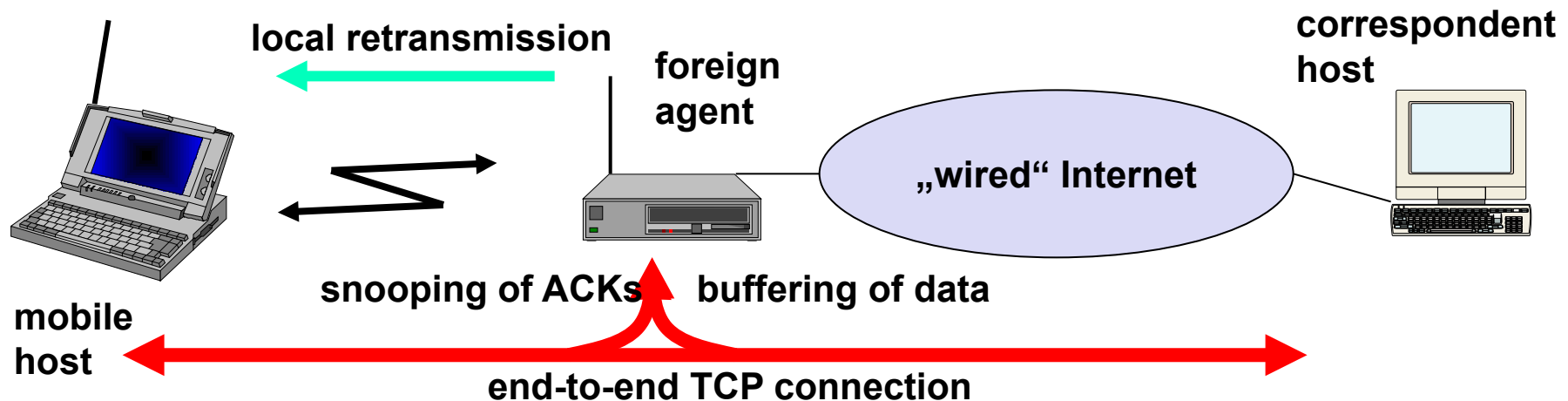
A handover between access points requires the migration of the TCP sockets and the TCP state (buffers, etc.)!

# Indirect TCP – Discussion

- Advantages
  - no changes in the fixed network necessary, no changes for the hosts (TCP protocol) necessary, all current optimizations to TCP still work
  - transmission errors on the wireless link do not propagate into the fixed network
  - simple to control, mobile TCP is used only for one hop between, e.g., a foreign agent and mobile host
  - therefore, a fast retransmission of packets is possible
- Disadvantages
  - **loss of end-to-end semantics**, an acknowledgement to a sender does not any longer mean that a receiver really got a packet, e.g. wireless link may drop or foreign agent might crash
  - **higher latency** possible due to buffering of data within the foreign agent and forwarding of state during handover
  - access point needs to be involved in L3 (e.g. IPsec) security mechanisms

# Snooping TCP – Principle

- „Transparent“ extension of TCP within the foreign agent
  - buffering of packets sent to the mobile host
  - lost packets on the wireless link (both directions!) will be retransmitted immediately by the mobile host or foreign agent, respectively (so called “local” retransmission)
  - the foreign agent therefore “snoops” the packet flow and recognizes acknowledgements in both directions, it also filters ACKs
  - changes of TCP only within the foreign agent



# Snooping TCP

- Data transfer to the mobile host (downlink)
  - FA buffers data until it receives ACK from the MH
  - FA detects packet loss on wireless link via timeouts (smaller timeout value than on CN) or DUPACKs from MH (which are discarded)
  - FA employs fast retransmission, transparent for the fixed network
- Data transfer from the mobile host (uplink)
  - FA detects packet loss on the wireless link via sequence numbers, FA answers directly with a NACK to the MH
  - MH can now retransmit data with only a very short delay
- Integration of the link layer
  - link layer often has similar mechanisms to those of TCP
- Problems
  - snooping TCP does not isolate the wireless link as good as I-TCP
  - snooping might be useless depending on encryption schemes, e.g. does not work with IPsec due to encryption of IP payload (including TCP segment number)

# Mobile TCP

- Special handling of lengthy and/or frequent disconnections
- M-TCP splits control as I-TCP does
  - unmodified TCP fixed network to supervisory host (SH)
  - optimized TCP between SH and MH (no slow start)
- Supervisory host (SH)
  - no caching, no retransmission (different from Indirect-TCP)
  - monitors all packets, if disconnection detected
    - set sender window size to 0
    - sender automatically goes into persistent mode
  - old or new SH reopens the window (set to old size) when reconnected
- Advantages
  - maintains semantics, supports disconnection, no buffer forwarding
- Disadvantages
  - loss on wireless link propagated into fixed network (no buffering)
  - adapted TCP on wireless link

## Transmission/timeout freezing

- Mobile hosts can be disconnected for a longer time
  - no packet exchange possible, e.g.,
    - discontinued communication in a tunnel
    - disconnection due to overloaded cells
    - preemption by higher priority traffic (scheduling)
  - TCP disconnects after time-out completely
- TCP freezing
  - PHY/MAC layer is often able to detect interruption in advance
  - PHY/MAC informs TCP layer of upcoming loss of connection
  - TCP stops sending, but does now not assume a congested link
  - PHY/MAC layer signals again if reconnected
- Advantage: scheme is independent of data
- Disadvantage:
  - TCP on mobile host has to be changed
  - mechanism depends on lower layers
  - does not work for sudden disconnections



## Forced fast retransmit/fast recovery

- Change of foreign agent often results in packet loss
  - TCP reacts with slow-start although there is no congestion
- Forced fast retransmit
  - as soon as the mobile host has registered with a new foreign agent (Mobile IP), the **MH sends DUPACKs on purpose**
  - this forces the fast retransmit mode at the communication partners (instead of slow start)
  - additionally, the TCP on the MH is forced to continue sending with the actual window size and not to go into slow-start after registration
- Advantage
  - simple changes result in significant higher performance
- Disadvantage
  - focus on problems due to (fast) handover, not on temporarily poor wireless link quality
  - mix of Mobile IP and TCP, no transparent approach

# Selective retransmission

- TCP acknowledgements are often cumulative
  - ACK n acknowledges correct and in-sequence receipt of packets up to n
  - if single packets are missing quite often a whole packet sequence beginning at the gap has to be retransmitted (go-back-n), thus wasting bandwidth
- Selective retransmission as one solution
  - RFC2018 allows for acknowledgements of single packets, not only acknowledgements of in-sequence packet streams without gaps
  - sender can now retransmit only the missing packets
  - mechanism is supported by newer TCP implementations
- Advantage
  - much higher efficiency
- Disadvantage
  - more complex software in a receiver, more buffers needed at the receiver

# TCP improvements for wireless

The analysis in [Section 4.3](#) showed that by simply splitting the TCP connection, we got an improvement in TCP throughput because the round trip delay over the wireless network is lower than the end-to-end latency for the non-split connection. For the case when link conditions in the wireless network are so bad that performance boost caused by the lower latency in the access is overwhelmed by the performance impact of link losses, we need to modify TCP2 to combat the link-specific problems that exist there. Note that these enhancements can also be implemented for the non-split TCP connection, which will also result in improvement in its performance. In practice, though, it may be easier to deploy these changes locally for TCP2 at the gateway node because this is under the operator's control.

The suggestions that have been made to improve TCP performance in the presence of wireless link impairments fall into the following areas:

1. Using available bandwidth estimates (ABEs) to estimate transmit rates: The TCP sender does not keep an estimate of the bottleneck link bandwidth; as a result, when it encounters congestion, it simply cuts down its window size by half. However, if the sender were able to keep an accurate estimate of the bottleneck link bandwidth (or ABE), then it can reduce its window size in a more intelligent fashion. This not only helps to reduce convergence time for the algorithm to the correct link rate, but also if the packet loss occurs because of link errors (rather than congestion), then the ABE rate is not reduced, which is reflected in the fact that the resulting sending rate remains high. As a result, these types of algorithms exhibit better performance in the presence of wireless link losses.
2. Loss Discrimination Algorithm (LDA): On detection of packet loss, if the TCP sender were given the additional information by an LDA about whether the loss was caused by congestion or link errors, then it can improve the congestion control algorithm by reducing its window size only if the loss was caused by congestion and simply retransmitting the missing packet if the loss was caused by link errors.
3. ZRW ACKs: In case the wireless link gets disconnected over extended periods of time, the TCP sender can undergo multiple time-outs, which will prevent transmission from restarting even after the link comes back up. To prevent this from occurring, the TCP receiver (or some other node along the path) can shut down the sender by sending a TCP ACK with a zero size receive window, which is known as a ZRW ACK.

# Content

- **Internet Protocol Suite**
- **TCP**
- **TCP over Wireless**
- **Beyond TCP**
  - Limitations of TCP
  - SPDY
  - QUIC
  - Multipath TCP

# Beyond TCP

Questions to be answered:

- What are the advantages of moving congestion control mechanisms into the user space?
- What does HoL blocking mean?
- What is the benefit of parallel multiple data streams?
- What is a bufferbloat?
- What are incast issues?
- What are the benefits of SPDY?
- What are the benefits of QUIC?
- What are the benefits of MPTCP?

# Beyond TCP

Limits of TCP over wireless:

- TCPs abstraction of the end-to-end connection, i.e. the underlying model, is poor, especially where wireless links are involved, resulting in poor performance
- TCP can not be easily adapted due to its implementation in the OS kernels and as all IP-based systems rely on it
- TCP does not support connections over multiple paths to increase reliability or throughput

*& mobility*

*↳ Multihoming*

Ideas

- ⇒ Solve transport functions in user space for flexibility and easy adaptability
  - e.g. to support adaptive congestion control mechanisms based on machine learning methods
- ⇒ Adapt to the needs of the application as well as the network
  - e.g. to support multihoming (mobility) and multistreaming (HoL problem)

## Beyond TCP – Research Trends

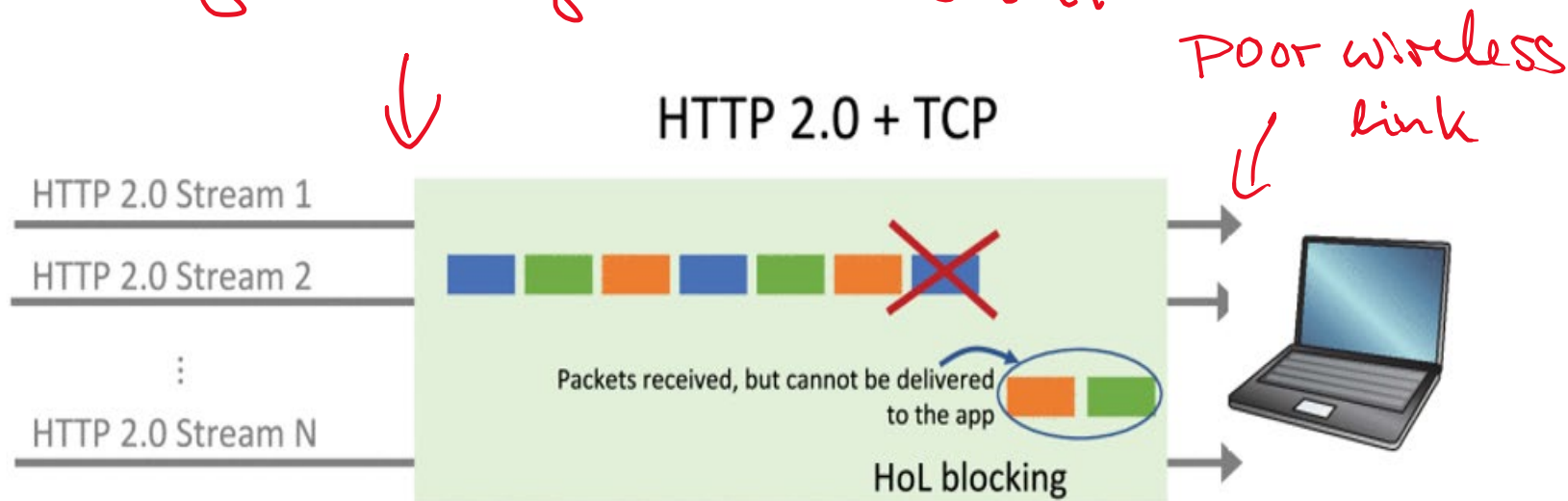
- Evolution of congestion control algorithms, to target optimal performance in challenging scenarios, possibly with the application of machine learning techniques *not covered here!*
- New transport protocols, alternative to the TCP implemented in the user-space e.g. SPDY and QUIC to improve latency
- Introduction of multipath capabilities at the transport layer, e.g. Multipath TCP, to support handovers, improve reliability or other issues

## Other TCP/IP Limitations

### Latency & Head of Line blocking (HoL)

- HoL arises due to in-order delivery requirement that creates delays when a packet is lost
- Higher latency when multiple independent data flows share same TCP connection due to in-order-delivery of TCP

*Sequencing on a single TCP conn.*



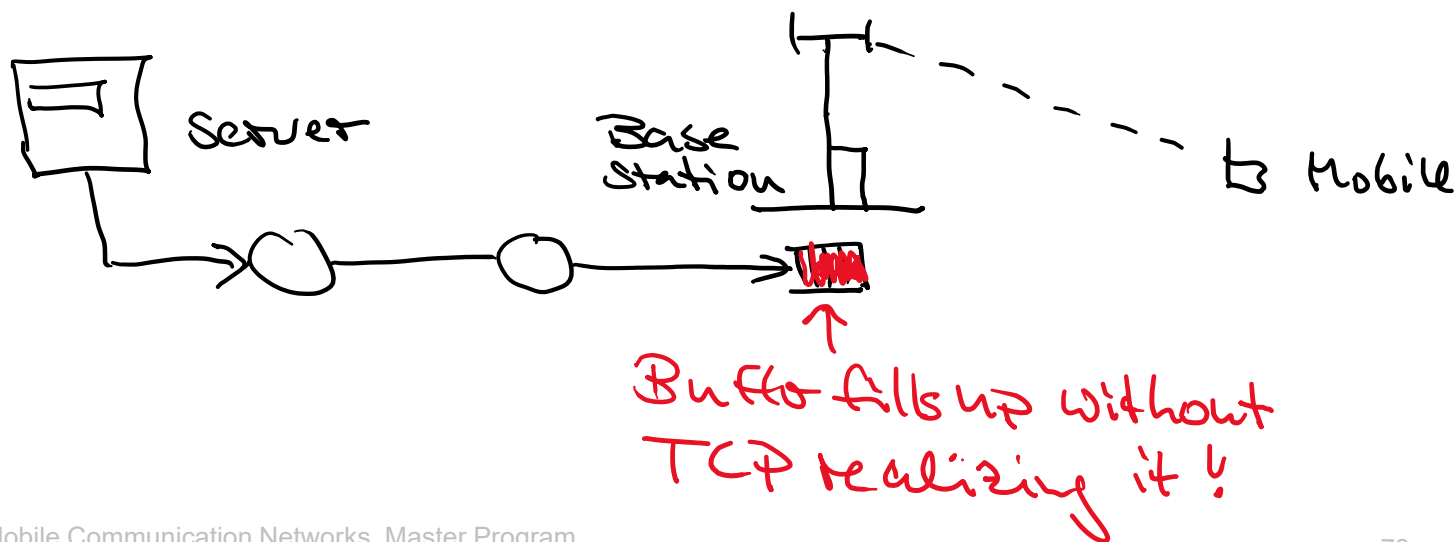
*Blocking http!*



# Other TCP/IP Limitations

## Bufferbloat

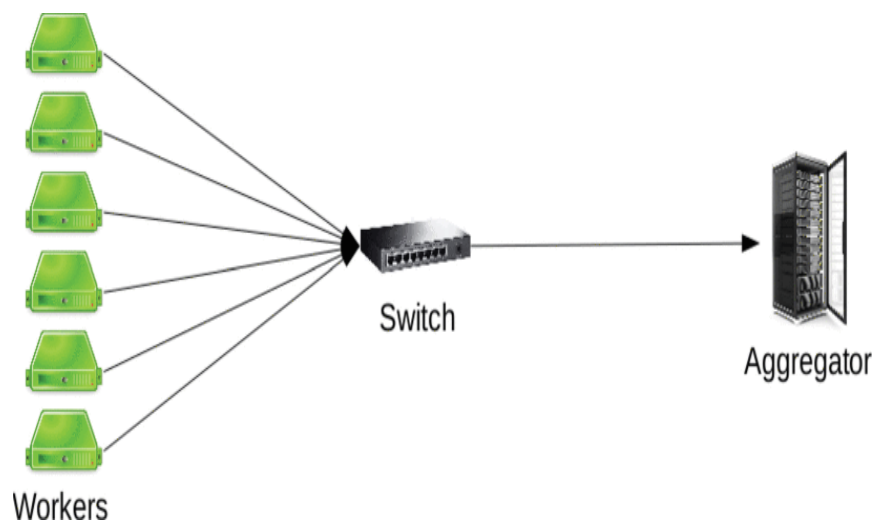
- Problem due to large buffers deployed in front of overloaded links
- Large buffers delay (TCPs) congestion mechanisms to learn about the overload and to slow down their data rate
- Without QoS mechanism in place this introduces increased delay and delay jitter especially harmful for latency-sensitive traffic as VoIP, gaming, etc.
- Problem may happen on wireless links in case that users or buffers are served in a non-monotonic way, e.g. to optimize system capacity (opportunistic scheduling)



# Other TCP/IP Limitations

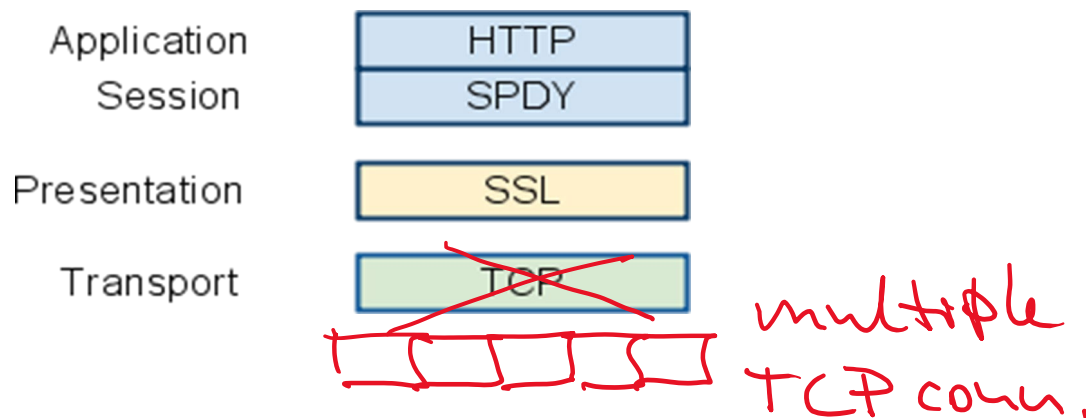
## Incast

- Problem with data centers/cloud computing frameworks where data are received from multiple sources and consumed by the client in a sequential (barrier-kind of) fashion
- Many-to-one simultaneous traffic flows create congestion and packet drops at a single node, e.g. switches, resulting in long delays
- Due to sequential consumption at the receiver, the delayed flow slows down consumption of all other flows so that the whole communication almost comes to a hold
- Measurements show a slowdown of the throughput up to several orders of magnitude!



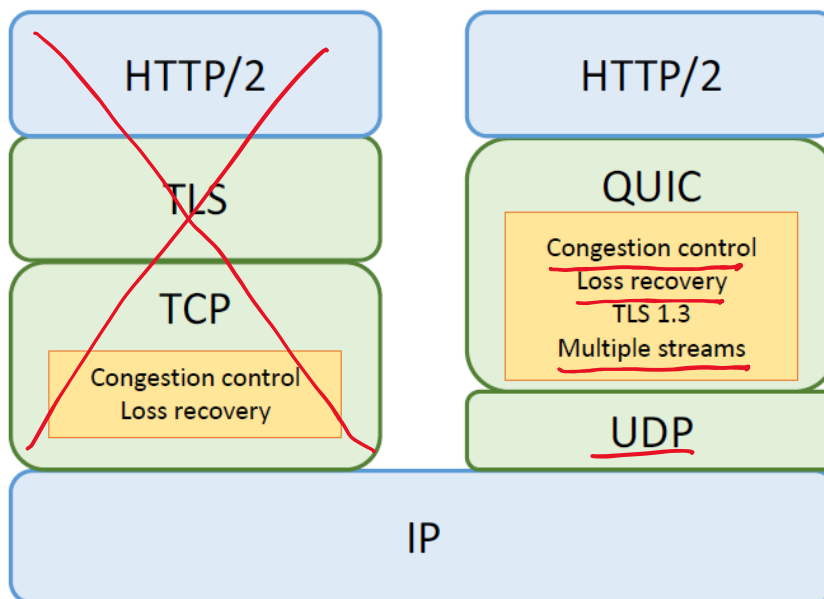
# SPeeDY (SPDY) protocol

- New application layer transport protocol running over TCP
- Targets reduced latency – fast access to web pages
- Multiplexes several data streams (requests) over several TCP connections – solves HoL problems of TCP
- Reduces required bandwidth through header compression – eliminates unnecessary headers
- New advanced feature: server initiated streams (“Server push” and “Server hint”)
- TCP Handshake still exists



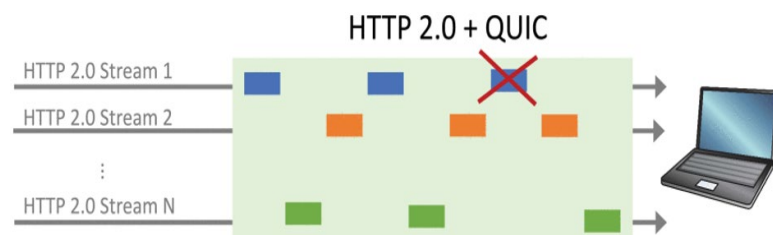
# Quick UDP Internet Connections (QUIC)

- New application layer transport protocol to overcome TCP limitations
- Handles congestion and flow control in application layer / user space
- Relies on UDP as underlying transport protocol – encapsulates packets into UDP datagrams
- Introduced by Google and currently under IETF standardization



# QUIC Modifications

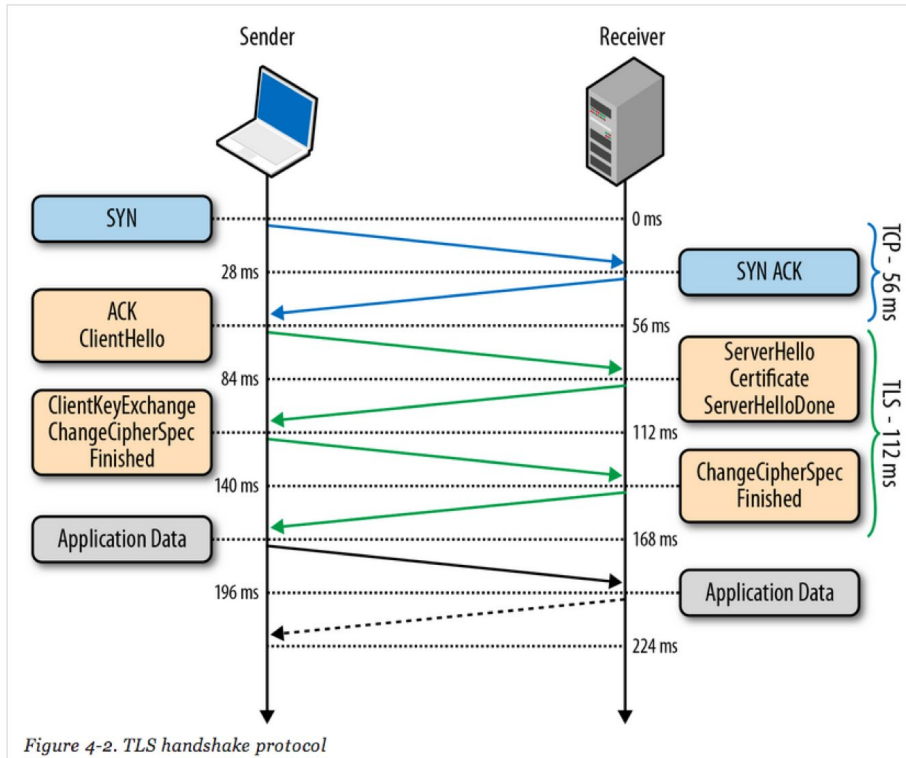
- Supports multiple data stream flows to solve HoL issues of TCP
  - Adds packet sequence numbers to the not in-order UDP packets
  - A lost QUIC packet doesn't stop other subsequent packets from being decrypted
- Congestion Control
  - Handled within user space
  - Uses loss-based mechanisms like in TCP
- Packet level error correction using FEC
  - Runs “exclusive or” (XOR) operation on some data packets
  - Sends XOR as the redundant error correction packet
  - Packet errors corrected at the receiver - no packet retransmissions



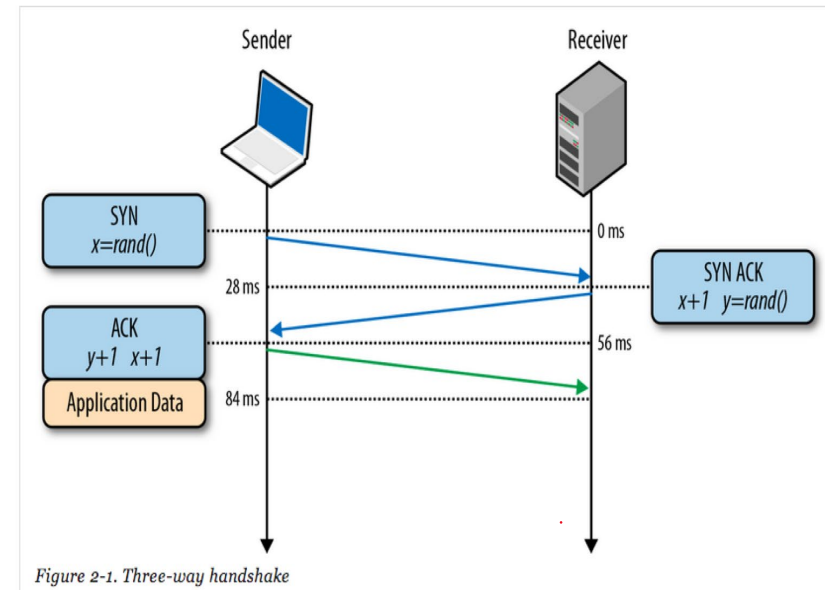
⇒ Research on improvements

# QUIC – Connection Establishment

## TCP + TLS



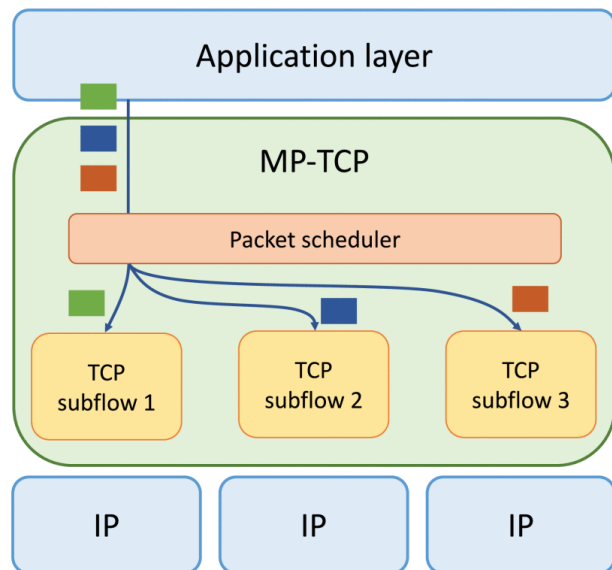
## QUIC



- TCP requires several RTTs before data exchange: three-way handshake  
⇒ Higher latency
- QUIC only requires a single RTT before data exchange

# Multipath TCP

- Newer devices support simultaneous use of multiple communication technologies, e.g. Wi-Fi and LTE
- Enhances system capacity, latency, reliability and reduces load on congested paths
- Handles congestion on each path (TCP subflow) independently
- Requires proper scheduling and congestion control mechanisms



*multiple application objects  
are each handled by an  
independent TCP flow*



*multiple flows may  
be handled by different  
interfaces (technologies)*



# Comparison of approaches for improved transport

| Approach                              | Mechanism                                                   | Advantages                                                                    | Disadvantages                                                                   |
|---------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Indirect TCP                          | splits TCP connection into two connections                  | isolation of wireless link, simple                                            | loss of TCP semantics, higher latency at handover                               |
| Snooping TCP                          | “snoops” data and acknowledgements, local retransmission    | transparent for end-to-end connection, MAC integration possible               | problematic with encryption, bad isolation of wireless link                     |
| M-TCP                                 | splits TCP connection, chokes sender via window size        | maintains end-to-end semantics, handles long term and frequent disconnections | bad isolation of wireless link, processing overhead due to bandwidth management |
| Transmission/timeout freezing         | freezes TCP state at disconnect, resumes after reconnection | independent of content or encryption, works for longer interrupts             | changes in TCP required, L1/2 dependant                                         |
| Forced fast retransmit/ fast recovery | avoids slow-start after roaming                             | simple and efficient                                                          | mixed layers (MIP), not transparent                                             |
| Selective retransmission              | retransmit lost data only                                   | very efficient                                                                | slightly more complex receiver software, more buffers needed                    |
| SPDY                                  | Multiple data streams                                       | Reduced latency                                                               |                                                                                 |
| QUIC                                  | User Space congestion control, multiple data streams        | Reduced latency, easier adaptations                                           |                                                                                 |
| MPTCP                                 | Connection via multiple technologies                        | Macro diversity                                                               | More complex tuning                                                             |



# References

## Literature on TCP:

- Ramjee Prasad, Marina Ruggieri: Technology Trends in Wireless Communications, Artech House, 2003
- W. Richard Stevens, “TCP/IP Illustrated, Volume 1: The Protocols“
- Douglas E. Comer: Computer Networks und Internets. 3. Edition, Prentice Hall, 2001

## Literature on TCP over wireless:

- Tian, Xu, Ansari: “TCP in Wireless Environments: Problems and Solutions”, IEEE Communications Magazine, 43(3), March 2005
- Balakrishnan et al, “A comparison of mechanisms for improving TCP performance over wireless links”, IEEE/ACM Transactions on Networking, Dec. 1997
- Xylomenos et al, “TCP performance issues over wireless links”, IEEE Communications Magazine, April 2001
- Balakrishnan et al, “How network asymmetry affects TCP”, IEEE Communications Magazine, April 2001
- Jochen Schiller: Mobile Communications (German and English), Addison-Wesley, 2005 (chapter 9 provides an overview on different approaches)

## Beyond TCP

- M. Polese, F. Chiariotti, E. Bonetto, F. Rigotto, A. Zanella and M. Zorzi, "A Survey on Recent Advances in Transport Layer Protocols," in *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3584-3608, Fourthquarter 2019.
- Gratzer, Florian. "Quic-quick udp internet connections." *Future Internet and Innovative Internet Technologies and Mobile Communications* (2016).
- Iyengar, Jana. "QUIC-Redefining Internet Transport." *Google Inc* (2015).
- IETF QUIC Specification, Draft 27: <https://tools.ietf.org/html/draft-ietf-quic-transport-27>